Diseño automatizado de zapatas aisladas según ACI 318-19

Diseño de Cimentaciones · Tarea 01 · Estudiante: Daysi Velecela · Noviembre 2026

1. Objetivo y alcance

El script `Diseno_Zapatas_Aisladas.py` (Python 3, numpy, pandas, matplotlib) lee las reacciones en la base exportadas de ETABS (*InputData01.xlsx*, hoja *Tarea1*: 13 apoyos × 6 combinaciones mayoradas DCON1–DCON6) y diseña cada zapata aislada con columna centrada de 60 × 50 cm. Las dimensiones en planta B_x, B_y son dato (provienen del dimensionamiento con cargas de servicio y presión admisible); el programa determina el peralte h y el armado, y verifica flexión, cortante unidireccional, punzonamiento, aplastamiento y anclaje. Los cortantes F_X, F_Y los toman las cadenas de amarre (criterio de clase) y no se trasladan como momento V·h. Se acompaña de una interfaz de escritorio y una versión web; el código está en *github.com/carodaysi/disenio-zapatas*.

2. Algoritmo

1. **Lectura y validación.** Se comprueban la hoja, las columnas obligatorias (Point, Load, FZ, MX, MY, Bx, By, Bcx, Bcy), las unidades de los encabezados (Bx[m]; un Bx[cm] se rechaza), celdas vacías o con texto (se indica la fila de Excel), dimensiones que parecen estar en cm, geometría que cambia entre combinaciones, columna mayor que la zapata y combinaciones repetidas. También se validan los parámetros de la sección 0. Cualquier error detiene el cálculo con un mensaje claro.
2. **Presión de contacto por combinación.** Zapata rígida sobre una malla de 200 × 200 celdas. Con contacto total, $q = P/A \pm M_y \cdot x/I_y \pm M_x \cdot y/I_x$ ($e_x = MY/P$, $e_y = MX/P$). Si aparece tracción, el plano de presiones se resuelve por Newton-Raphson sin tracción en el suelo (equilibrio de P, M_x, M_y). Cada combinación queda con su estado: *OK*, *contacto parcial* (zapata en REVISAR) o *resultante fuera de la zapata / no converge* (NO CUMPLE).
3. **Peso propio.** $W = 0.9 \cdot (\gamma_c \cdot h + \gamma_s \cdot (D_f - h)) \cdot B_x \cdot B_y$ se suma a P para el contacto y se recalcula con cada h; flexión y cortante usan la presión neta $q_u - w$, de modo que W no se cuenta dos veces.
4. **Iteración del peralte.** h = 40, 45, 50 … 150 cm. Para cada h se prueban los diámetros Ø12, 14, 16 … y se toma el primero con separación ≥ 10 cm y anclaje dentro del voladizo.
5. **Verificaciones para el h de prueba** (todas las combinaciones resueltas): M_u en la cara, A_s requerido y mínimo, barras y φM_n; V_u a d de la cara y φV_c; V_u en el perímetro a d/2 con transferencia de momento y φv_c. Se calcula D/C de cada verificación y combinación; el h final es el primero con todos los D/C ≤ 1, y se registra qué verificación obligó a subirlo (*Controla h*). Si d ≥ voladizo, la zapata trabaja como hormigón en masa y el cortante no aplica.
6. **Complementos y salidas.** Aplastamiento columna-zapata, anclaje (recto o con gancho), distribución en zapatas rectangulares y acero superior si hay levantamiento. Se generan la memoria de cálculo, un Excel (Resumen, Detalle_Combos, Datos_Entrada, Parametros) y figuras.

3. Criterios ACI 318-19

f'c = 210 kg/cm², fy = 4200 kg/cm², $\lambda = \lambda_s = 1$ (13.2.6.2), recubrimiento 7.5 cm (20.5.1.3.1), γ_c = 2.4 t/m³, γ_s = 1.8 t/m³, D_f = 1.5 m. Unidades: t, m (esfuerzos en kg/cm²). Criterio por defecto `CRITERIO = "ACI"`:

Verificación	Expresión / criterio	ACI 318-19
Peralte efectivo	$d = h - rec - d_b/2$ (capa inferior, dirección de mayor M_u); $d = h - rec - 1.5 \cdot d_b$ (capa superior); d ≥ 15 cm	13.3.1.2
Flexión	M_u en la cara de la columna; $\varphi M_n = \varphi \cdot A_s \cdot f_y \cdot (d - a/2)$, φ = 0.90 comprobando ε_t; $A_{s,mín} = 0.0018 \cdot b \cdot h$; s ≤ mín(2h, 45 cm); s libre ≥ 2.5 cm	13.2.7.1, 21.2.2, 8.6.1.1, 8.7.2.2, 25.2.1
Cortante 1D	Sección a d de la cara; $V_c = 2.12 \cdot \lambda_s \cdot \lambda \cdot \rho_w^{1/3} \cdot \sqrt{f'c} \cdot b_w \cdot d \le 1.33 \cdot \lambda \cdot \sqrt{f'c} \cdot b_w \cdot d$; φ = 0.75	13.2.7.2, 22.5.5.1
Punzonamiento	Perímetro b_o a d/2; $v_c = mín[1.06;\ 0.53(1 + 2/\beta);\ 0.27(2 + \alpha_s d/b_o)] \cdot \lambda_s \cdot \lambda \cdot \sqrt{f'c}$, $\alpha_s = 40$; $v_u = V_u/(b_o d) + \gamma_v \cdot M_{sc} \cdot c/J_c$	22.6.4.1, Tabla 22.6.5.2, 8.4.4.2
Aplastamiento	$\varphi B_n = 0.65 \cdot 0.85 \cdot f'c \cdot A_1 \cdot \sqrt{(A_2/A_1)}$, $\sqrt{(A_2/A_1)} \le 2$	22.8.3.2
Anclaje y zapata rectangular	ℓ_d recto o ℓ_{dh} con gancho dentro del voladizo; banda central $\gamma_s = 2/(\beta + 1)$	25.4.2.3, 25.4.3.1, 13.3.3.3

Criterio de aceptación: CUMPLE si todos los D/C ≤ 1 y todas las combinaciones mantienen contacto total; REVISAR si alguna combinación pierde contacto parcialmente (la resistencia se verifica con la presión resuelta, pero B_x, B_y deben revisarse); NO CUMPLE si falta resistencia hasta h = 150 cm o alguna resultante sale de la zapata.

4. Contraste con el Excel de clase

Se usó la hoja *Empotrado Art* del archivo *Dimensionamiento.xlsx* desarrollado en clase (23/10/2026). Parte de las mismas reacciones DCON del modelo ETABS; las zapatas P4, P5 y P6 tienen el mismo B que en la tarea (2.30, 2.10 y 2.60 m), por lo que el contraste es directo. El programa se evaluó con el mismo h y la misma barra Ø14 del Excel; las magnitudes por combinación corresponden a la combinación que gobierna (DCON5 en P5, DCON4 en P6).

Magnitud	P5 Excel	P5 Prog.	Δ %	P6 Excel	P6 Prog.	Δ %
B × B [m] · h [cm]	2.10 · 45	2.10 · 45		2.60 · 60	2.60 · 60	
Mu,x / Mu,y [t·m]	15.81	15.81	0.0	33.65	33.65	0.0
	18.47	18.47	0.0	36.64	36.64	0.0
d x / d y [cm]	37.5	35.4	-5.6	52.5	50.4	-4.0
	37.5	36.8	-1.9	52.5	51.8	-1.3
As,mín [cm²]	14.18	17.01	+20.0	24.57	28.08	+14.3
As provisto [cm²]	23.09 (15Ø14)	18.47 (12Ø14)	-20.0	27.71 (18Ø14)	29.25 (19Ø14)	+5.6
φMn,y [t·m]	31.60	24.97	-21.0	53.67	55.81	+4.0
Vu 1D x / y [t]	21.09	22.27	+5.6	32.01	33.42	+4.4
	24.58	24.98	+1.6	34.90	35.36	+1.3
φVc 1D y [t]	25.97	23.81	-8.3	39.82	40.19	+0.9
D/C cortante 1D y	0.946	1.049	+10.9	0.876	0.880	+0.4
Vu punzonamiento [t]	94.62	95.31	+0.7	143.27	144.04	+0.5
bo [cm]	370.0	364.4	-1.5	430.0	424.4	-1.3
φVc punzonamiento [t]	165.88	151.55	-8.6	269.88	249.85	-7.4
D/C punzonamiento	0.570	0.646	+13.3	0.531	0.582	+9.7
h final de diseño [cm]	**45**	**50**		**60**	**60**	

φVc de punzonamiento del Excel: $0.75 \cdot v_{c1} \cdot b_o \cdot d$ con $v_{c1} = 1.1\sqrt{f'c} = 15.94$ kg/cm² (para P5 coincide con el valor de la hoja, 165.88 t). En P4 (h = 50 cm) el resultado es análogo: Mu 22.37/24.64 t·m en ambos y el programa sube a h = 55 cm por cortante 1D.

Justificación de las discrepancias.

- **Solicitaciones (M_u, V_u de punzonamiento): coinciden (Δ ≤ 0.7 %).** Ambos usan la presión lineal P/A ± M·c/I (contacto total en estas combinaciones); el Excel la evalúa sobre los ejes y el programa la integra en toda la planta, lo que con momentos pequeños da el mismo resultado.
- **Peralte efectivo.** El Excel toma d = H − 7.5 cm en las dos direcciones; el programa descuenta medio diámetro y, en la capa superior, uno y medio (d_x 2.1 cm y d_y 0.7 cm menores). Con d menor la sección crítica se acerca a la columna y V_u 1D crece entre 1 % y 6 %; φV_c baja.
- **Acero mínimo y provisto.** El Excel usa $A_{s,mín} = 0.0018 \cdot B \cdot d$; el programa $0.0018 \cdot B \cdot h$, como pide 8.6.1.1 (sobre A_g). El Excel coloca barras a 15 cm fijos, con más acero del requerido (P5: 23.09 cm² frente a $A_{s,req} \approx 13.6$ cm²); el programa coloca máx($A_{s,req}$, $A_{s,mín}$). Como V_c crece con $\rho_w^{1/3}$, el Excel obtiene mayor φV_c. Con el A_s y d del Excel, las funciones del programa dan exactamente φMn = 31.60 t·m y φVc = 25.97 t en P5 (Excel: 31.60 y 25.97): las fórmulas son las mismas.
- **Punzonamiento.** El Excel usa $v_c = 1.1\sqrt{f'c}$; el ACI 318-19 en kg/cm² da $0.33\sqrt{f'c}$ [MPa] = $1.054\sqrt{f'c} \approx 1.06\sqrt{f'c}$, valor del programa. El programa suma además el término $\gamma_v \cdot M_{sc} \cdot c/J_c$, por lo que su D/C es algo mayor (más conservador).
- **Consecuencia.** En P6 ambos llegan a h = 60 cm. En P5 el programa, con su armado mínimo, obtiene D/C de cortante 1D = 1.049 con h = 45 cm y sube a 50 cm; el Excel cumple con 45 cm porque el acero adicional eleva V_c. Es una decisión de diseño (más peralte o más acero), no un error de cálculo.

Reproducción exacta del ejemplo de clase. Para validar el algoritmo, el parámetro `CRITERIO = "CLASE"` adopta las hipótesis del Excel (d = h − 7.5 cm, $A_{s,mín} = 0.0018 \cdot b \cdot d$, barras a 15 cm, $v_{c1} = 1.1\sqrt{f'c}$, sin transferencia de momento). Con él, el programa itera h y armado desde 40 cm y obtiene los valores de la hoja en las cinco zapatas (datos de la hoja de clase; en P7 el Excel usa Ø16, elegido a mano, que se impone con `DIAMETRO_PUNTO = {7: 16}`). El diseño entregado (Anexo B) usa el criterio ACI.

Zapata	h [cm] Excel | Prog.	Barras Excel | Prog.	As [cm²] Excel | Prog.	φMn [t·m] Excel | Prog.	φVc 1D [t] Excel | Prog.	Vu punz. [t] Excel | Prog.
P3 (2.40 m)	45 | 45	17Ø14 | 17Ø14	26.17 | 26.17	35.83 | 35.83	29.60 | 29.60	100.98 | 100.95
P4 (2.30 m)	50 | 50	16Ø14 | 16Ø14	24.63 | 24.63	38.40 | 38.40	30.65 | 30.65	114.78 | 114.83
P5 (2.10 m)	45 | 45	15Ø14 | 15Ø14	23.09 | 23.09	31.60 | 31.60	25.97 | 25.97	94.62 | 94.61
P6 (2.60 m)	60 | 60	18Ø14 | 18Ø14	27.71 | 27.71	53.67 | 53.67	39.82 | 39.82	143.27 | 143.24
P7 (3.00 m)	65 | 65	21Ø16 | 21Ø16	42.22 | 42.22	89.13 | 89.13	53.57 | 53.57	204.18 | 204.17

5. Resultados y control de combinaciones

De las 13 zapatas, **11 cumplen** y **2 quedan en REVISAR**. P3–P13 requieren h = 50–70 cm; en todas gobierna el cortante 1D en Y (D/C 0.81–0.997), con punzonamiento D/C ≤ 0.64 y flexión D/C ≤ 0.76. Las combinaciones sísmicas DCON3–DCON6 controlan todas las verificaciones. P1 y P2 (1.10 × 1.10 m) quedan en h = 40 cm con d ≥ voladizo (hormigón en masa, sin cortante ni punzonamiento) pero pierden contacto parcial con el suelo en DCON5 (99.0 %) y DCON6 (76.6 %): su resistencia está verificada con la presión sin tracción, y se recomienda revisar B por estabilidad y presión admisible. Ninguna combinación saca la resultante de la zapata. El detalle por zapata está en el Anexo B; los datos de entrada en el Anexo A y el script en el Anexo C.

Anexo A. Datos de entrada (InputData01.xlsx, hoja Tarea1)

Reacciones en la base para las combinaciones de diseño mayoradas (t, t·m) y geometría (m). Story = BASE en todas las filas. MZ no interviene en el diseño de la zapata.

Point	Load	FX [t]	FY [t]	FZ [t]	MX [t·m]	MY [t·m]	MZ [t·m]	Bx [m]	By [m]	Bcx [m]	Bcy [m]
1	DCON1	0.06	0.22	12.06	-0.053	0.012	0.007	1.10	1.10	0.60	0.50
1	DCON2	0.07	0.23	11.15	-0.055	0.013	0.006	1.10	1.10	0.60	0.50
1	DCON3	-7.23	0.49	5.20	-0.117	-1.411	-0.110	1.10	1.10	0.60	0.50
1	DCON4	7.37	0.00	26.31	-0.001	1.438	0.124	1.10	1.10	0.60	0.50
1	DCON5	0.00	-7.63	6.49	1.812	-0.001	-1.235	1.10	1.10	0.60	0.50
1	DCON6	0.15	8.13	18.65	-1.930	0.028	1.249	1.10	1.10	0.60	0.50
2	DCON1	0.05	-0.24	11.96	0.056	0.010	-0.009	1.10	1.10	0.60	0.50
2	DCON2	0.05	-0.25	11.05	0.059	0.011	-0.011	1.10	1.10	0.60	0.50
2	DCON3	-7.07	-0.01	5.50	0.003	-1.379	-0.144	1.10	1.10	0.60	0.50
2	DCON4	7.19	-0.52	26.42	0.122	1.402	0.122	1.10	1.10	0.60	0.50
2	DCON5	1.68	-8.12	21.36	1.926	0.327	-1.162	1.10	1.10	0.60	0.50
2	DCON6	-1.56	7.59	3.56	-1.800	-0.305	1.140	1.10	1.10	0.60	0.50
3	DCON1	0.05	0.09	59.11	-0.022	0.010	0.001	2.60	2.60	0.60	0.50
3	DCON2	0.07	0.12	61.78	-0.028	0.013	0.001	2.60	2.60	0.60	0.50
3	DCON3	-6.13	0.38	25.20	-0.091	-1.196	-0.129	2.60	2.60	0.60	0.50
3	DCON4	6.26	-0.15	106.91	0.035	1.222	0.131	2.60	2.60	0.60	0.50
3	DCON5	-1.35	-7.35	13.57	1.745	-0.263	-1.012	2.60	2.60	0.60	0.50
3	DCON6	1.48	7.59	118.54	-1.802	0.289	1.014	2.60	2.60	0.60	0.50
4	DCON1	0.10	0.11	98.66	-0.027	0.019	-0.002	2.30	2.30	0.60	0.50
4	DCON2	0.13	0.13	108.51	-0.030	0.026	-0.003	2.30	2.30	0.60	0.50
4	DCON3	-6.41	0.39	87.40	-0.093	-1.250	-0.132	2.30	2.30	0.60	0.50
4	DCON4	6.66	-0.13	139.85	0.030	1.300	0.127	2.30	2.30	0.60	0.50
4	DCON5	0.06	-8.02	121.70	1.902	0.011	-1.142	2.30	2.30	0.60	0.50
4	DCON6	0.20	8.28	105.55	-1.965	0.039	1.137	2.30	2.30	0.60	0.50
5	DCON1	0.03	-0.23	66.49	0.054	0.006	0.000	2.10	2.10	0.60	0.50
5	DCON2	0.05	-0.27	70.04	0.065	0.009	-0.001	2.10	2.10	0.60	0.50
5	DCON3	-6.19	-0.01	48.91	0.003	-1.207	-0.118	2.10	2.10	0.60	0.50
5	DCON4	6.27	-0.54	100.38	0.128	1.224	0.118	2.10	2.10	0.60	0.50
5	DCON5	1.74	-7.67	117.32	1.821	0.340	-1.167	2.10	2.10	0.60	0.50
5	DCON6	-1.66	7.12	31.97	-1.689	-0.323	1.166	2.10	2.10	0.60	0.50
6	DCON1	-0.08	0.16	83.29	-0.037	-0.015	-0.001	2.60	2.60	0.60	0.50
6	DCON2	-0.09	0.20	89.34	-0.048	-0.019	-0.002	2.60	2.60	0.60	0.50
6	DCON3	-6.96	0.19	16.27	-0.045	-1.357	-0.116	2.60	2.60	0.60	0.50
6	DCON4	6.77	0.21	172.74	-0.049	1.320	0.113	2.60	2.60	0.60	0.50
6	DCON5	-1.63	-8.67	31.81	2.059	-0.317	-1.111	2.60	2.60	0.60	0.50
6	DCON6	1.44	9.07	157.21	-2.153	0.280	1.108	2.60	2.60	0.60	0.50
7	DCON1	-0.13	0.07	134.46	-0.017	-0.025	0.000	2.85	2.85	0.60	0.50
7	DCON2	-0.16	0.09	152.52	-0.022	-0.031	0.000	2.85	2.85	0.60	0.50
7	DCON3	-6.72	0.10	77.99	-0.023	-1.311	-0.117	2.85	2.85	0.60	0.50
7	DCON4	6.40	0.09	237.52	-0.020	1.249	0.117	2.85	2.85	0.60	0.50
7	DCON5	-0.18	-9.54	175.45	2.264	-0.036	-1.068	2.85	2.85	0.60	0.50
7	DCON6	-0.14	9.73	140.06	-2.308	-0.027	1.068	2.85	2.85	0.60	0.50
8	DCON1	0.08	-0.25	110.23	0.058	0.016	-0.002	2.50	2.50	0.60	0.50
8	DCON2	0.11	-0.32	123.38	0.075	0.022	-0.004	2.50	2.50	0.60	0.50
8	DCON3	-5.80	-0.30	146.25	0.070	-1.131	-0.125	2.50	2.50	0.60	0.50
8	DCON4	6.01	-0.33	110.33	0.078	1.173	0.118	2.50	2.50	0.60	0.50
8	DCON5	1.68	-9.15	171.04	2.171	0.328	-1.115	2.50	2.50	0.60	0.50
8	DCON6	-1.47	8.52	85.55	-2.023	-0.286	1.108	2.50	2.50	0.60	0.50
9	DCON1	0.02	0.16	84.74	-0.039	0.003	0.000	2.50	2.50	0.60	0.50
9	DCON2	0.02	0.21	91.23	-0.050	0.005	-0.001	2.50	2.50	0.60	0.50
9	DCON3	-6.88	0.08	162.85	-0.018	-1.342	-0.115	2.50	2.50	0.60	0.50
9	DCON4	6.92	0.34	29.87	-0.080	1.351	0.114	2.50	2.50	0.60	0.50
9	DCON5	-1.51	-9.40	42.67	2.232	-0.294	-1.080	2.50	2.50	0.60	0.50
9	DCON6	1.55	9.82	150.05	-2.330	0.303	1.079	2.50	2.50	0.60	0.50
10	DCON1	0.06	-0.14	118.75	0.034	0.012	0.000	2.85	2.85	0.60	0.50
10	DCON2	0.09	-0.18	134.57	0.044	0.017	-0.001	2.85	2.85	0.60	0.50
10	DCON3	-6.55	-0.31	208.13	0.074	-1.278	-0.110	2.85	2.85	0.60	0.50
10	DCON4	6.71	-0.05	70.35	0.012	1.310	0.109	2.85	2.85	0.60	0.50
10	DCON5	0.07	-9.83	190.99	2.333	0.013	-1.076	2.85	2.85	0.60	0.50
10	DCON6	0.10	9.47	87.49	-2.247	0.019	1.074	2.85	2.85	0.60	0.50
11	DCON1	-0.07	0.10	66.73	-0.024	-0.015	-0.001	2.20	2.20	0.60	0.50
11	DCON2	-0.09	0.12	69.56	-0.029	-0.018	-0.002	2.20	2.20	0.60	0.50
11	DCON3	-6.32	-0.23	126.63	0.054	-1.232	-0.099	2.20	2.20	0.60	0.50
11	DCON4	6.13	0.48	22.29	-0.113	1.196	0.096	2.20	2.20	0.60	0.50
11	DCON5	-1.50	-10.72	17.50	2.544	-0.293	-1.068	2.20	2.20	0.60	0.50

Point	Load	FX [t]	FY [t]	FZ [t]	MX [t·m]	MY [t·m]	MZ [t·m]	Bx [m]	By [m]	Bcx [m]	Bcy [m]
11	DCON6	1.32	10.97	131.42	-2.603	0.257	1.065	2.20	2.20	0.60	0.50
12	DCON1	-0.13	0.03	101.31	-0.007	-0.025	0.000	2.50	2.50	0.60	0.50
12	DCON2	-0.16	0.03	111.05	-0.008	-0.032	-0.001	2.50	2.50	0.60	0.50
12	DCON3	-6.13	-0.33	168.14	0.078	-1.196	-0.116	2.50	2.50	0.60	0.50
12	DCON4	5.81	0.40	64.74	-0.095	1.133	0.115	2.50	2.50	0.60	0.50
12	DCON5	-0.19	-11.69	130.35	2.774	-0.037	-1.094	2.50	2.50	0.60	0.50
12	DCON6	-0.13	11.76	102.54	-2.790	-0.026	1.092	2.50	2.50	0.60	0.50
13	DCON1	-0.21	-0.13	76.15	0.032	-0.041	0.003	2.20	2.20	0.60	0.50
13	DCON2	-0.26	-0.17	82.34	0.041	-0.051	0.003	2.20	2.20	0.60	0.50
13	DCON3	-5.78	-0.53	108.12	0.126	-1.128	-0.107	2.20	2.20	0.60	0.50
13	DCON4	5.26	0.19	65.51	-0.045	1.026	0.114	2.20	2.20	0.60	0.50
13	DCON5	1.21	-10.94	132.99	2.597	0.236	-1.044	2.20	2.20	0.60	0.50
13	DCON6	-1.73	10.60	40.64	-2.516	-0.338	1.051	2.20	2.20	0.60	0.50

Anexo B. Resultados por zapata

f'c = 210 kg/cm² y fy = 4200 kg/cm² en todas las zapatas; columna 60 × 50 cm centrada. As en cm², Mu y φMn en t·m. X: barras paralelas a X (voladizo en X); Y: barras paralelas a Y. Armado: n Ø (mm) @ separación (cm), igual en malla inferior.

B.1 Geometría, materiales y flexión

Punto	Bx × By [m]	f'c [kg/cm²]	fy [kg/cm²]	h [cm]	dx [cm]	dy [cm]	Mu x	As req X	As mín X	As prov X	Armado X	φMn x	D/C flex X	Mu y	As req Y	As mín Y	As prov Y	Armado Y	φMn y	D/C flex Y
1	1.10 × 1.10	210	4200	40	31.0	32.0	0.94	0.80	7.92	8.64	11Ø10@9	9.82	0.095	1.12	0.92	7.92	8.64	11Ø10@9	10.15	0.110
2	1.10 × 1.10	210	4200	40	31.0	32.0	0.93	0.80	7.92	8.64	11Ø10@9	9.82	0.095	1.23	1.02	7.92	8.64	11Ø10@9	10.15	0.121
3	2.60 × 2.60	210	4200	50	40.7	41.9	22.89	15.13	23.40	23.75	21Ø12@12	35.57	0.643	25.78	16.57	23.40	23.75	21Ø12@12	36.65	0.703
4	2.30 × 2.30	210	4200	55	45.7	46.9	22.37	13.14	22.77	23.75	21Ø12@10	39.94	0.560	24.64	14.11	22.77	23.75	21Ø12@10	41.01	0.601
5	2.10 × 2.10	210	4200	50	40.7	41.9	15.81	10.43	18.90	19.23	17Ø12@12	28.80	0.549	18.47	11.85	18.90	19.23	17Ø12@12	29.67	0.623
6	2.60 × 2.60	210	4200	60	50.7	51.9	33.65	17.85	28.08	28.27	25Ø12@10	52.82	0.637	36.64	18.99	28.08	28.27	25Ø12@10	54.10	0.677
7	2.85 × 2.85	210	4200	70	60.4	61.8	53.17	23.67	35.91	36.95	24Ø14@11	82.22	0.647	57.54	25.05	35.91	36.95	24Ø14@11	84.18	0.684
8	2.50 × 2.50	210	4200	60	50.7	51.9	30.98	16.41	27.00	27.14	24Ø12@10	50.71	0.611	34.97	18.12	27.00	27.14	24Ø12@10	51.94	0.673
9	2.50 × 2.50	210	4200	55	45.7	46.9	29.83	17.59	24.75	24.88	22Ø12@11	41.88	0.712	32.58	18.73	24.75	24.88	22Ø12@11	43.01	0.757
10	2.85 × 2.85	210	4200	65	55.4	56.8	46.65	22.66	33.34	33.87	22Ø14@12	69.13	0.675	50.44	23.91	33.34	33.87	22Ø14@12	70.92	0.711
11	2.20 × 2.20	210	4200	50	40.7	41.9	19.19	12.69	19.80	20.36	18Ø12@11	30.48	0.630	22.45	14.44	19.80	20.36	18Ø12@11	31.40	0.715
12	2.50 × 2.50	210	4200	60	50.7	51.9	30.74	16.28	27.00	27.14	24Ø12@10	50.71	0.606	33.66	17.43	27.00	27.14	24Ø12@10	51.94	0.648
13	2.20 × 2.20	210	4200	50	40.7	41.9	19.41	12.84	19.80	20.36	18Ø12@11	30.48	0.637	22.70	14.61	19.80	20.36	18Ø12@11	31.40	0.723

B.2 Cortante unidireccional, punzonamiento, combinación controlante y estado

Punto	Vu 1D X [t]	φVc X [t]	D/C 1D X	Comb.	Vu 1D Y [t]	φVc Y [t]	D/C 1D Y	Comb.	Vu punz. [t]	φVc punz. [t]	D/C punz.	Comb.	Verificación gobernante	Comb. control.	Estado
1	N/A	10.71	N/A	—	N/A	10.94	N/A	—	N/A	125.56	N/A	—	Flexión Y (D/C = 0.110)	DCON6	REVISAR (1)
2	N/A	10.71	N/A	—	N/A	10.94	N/A	—	N/A	125.56	N/A	—	Flexión Y (D/C = 0.121)	DCON5	REVISAR (1)
3	27.15	31.92	0.851	DCON6	29.53	32.55	0.907	DCON6	102.32	183.28	0.572	DCON6	Cortante 1D Y (D/C = 0.907)	DCON6	CUMPLE
4	24.38	31.78	0.767	DCON4	26.22	32.33	0.811	DCON4	112.79	216.14	0.529	DCON4	Cortante 1D Y (D/C = 0.811)	DCON4	CUMPLE
5	19.30	25.80	0.748	DCON5	22.06	26.31	0.838	DCON5	92.72	183.28	0.519	DCON5	Cortante 1D Y (D/C = 0.838)	DCON5	CUMPLE
6	33.22	39.17	0.848	DCON4	35.30	39.79	0.887	DCON4	143.93	251.30	0.578	DCON4	Cortante 1D Y (D/C = 0.887)	DCON4	CUMPLE
7	43.81	51.16	0.856	DCON4	46.43	51.95	0.894	DCON4	198.18	326.90	0.610	DCON4	Cortante 1D Y (D/C = 0.894)	DCON4	CUMPLE
8	30.42	37.64	0.808	DCON5	33.72	38.24	0.882	DCON5	140.19	251.30	0.568	DCON5	Cortante 1D Y (D/C = 0.882)	DCON5	CUMPLE
9	32.62	34.12	0.956	DCON3	34.60	34.72	0.997	DCON3	136.18	216.14	0.637	DCON3	Cortante 1D Y (D/C = 0.997)	DCON3	CUMPLE
10	42.13	46.92	0.898	DCON3	44.35	47.70	0.930	DCON3	176.57	287.22	0.619	DCON3	Cortante 1D Y (D/C = 0.930)	DCON3	CUMPLE
11	23.58	27.13	0.869	DCON6	26.86	27.66	0.971	DCON6	106.31	183.28	0.598	DCON6	Cortante 1D Y (D/C = 0.971)	DCON6	CUMPLE
12	30.21	37.64	0.803	DCON3	32.38	38.24	0.847	DCON3	137.81	251.30	0.554	DCON3	Cortante 1D Y (D/C = 0.847)	DCON3	CUMPLE
13	23.85	27.13	0.879	DCON5	27.17	27.66	0.982	DCON5	107.58	183.28	0.605	DCON5	Cortante 1D Y (D/C = 0.982)	DCON5	CUMPLE

N/A: no aplica, d ≥ voladizo (hormigón en masa; la sección crítica queda fuera de la zapata). (1) P1 y P2: pérdida de contacto parcial en DCON5 (99.0 % en contacto) y DCON6 (76.6 %); resistencia verificada con la presión sin tracción, revisar B por estabilidad y presión admisible.

B.3 Control del peralte, presiones y verificaciones complementarias

Punto	qu máx [t/m²]	Contacto mín [%]	W zapata + relleno [t]	Controla h	Aplast.	Anclaje	Acero superior
1	30.88	99.0	3.20	Peralte inicial de búsqueda (40 cm): cumple todas las verificaciones con el primer h probado	CUMPLE	Gancho estándar 90°	Paralelo a X: 11 Ø10 mm @ 9 cm; paralelo a Y: 11 Ø10 mm @ 9 cm
2	31.35	76.6	3.20	Peralte inicial de búsqueda (40 cm): cumple todas las verificaciones con el primer h probado	CUMPLE	Gancho estándar 90°	Paralelo a X: 11 Ø10 mm @ 9 cm; paralelo a Y: 11 Ø10 mm @ 9 cm
3	20.95	100.0	18.25	Con h = 45 cm no cumple: Cortante 1D X (D/C = 1.041, DCON6); Cortante 1D Y (D/C = 1.101, DCON6)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
4	29.82	100.0	14.43	Con h = 50 cm no cumple: Cortante 1D Y (D/C = 1.009, DCON4)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
5	30.70	100.0	11.91	Con h = 45 cm no cumple: Cortante 1D Y (D/C = 1.052, DCON5)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
6	28.77	100.0	18.62	Con h = 55 cm no cumple: Cortante 1D X (D/C = 1.029, DCON4); Cortante 1D Y (D/C = 1.068, DCON4)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
7	32.38	100.0	22.81	Con h = 65 cm no cumple: Cortante 1D X (D/C = 1.023, DCON4); Cortante 1D Y (D/C = 1.061, DCON4)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
8	31.08	100.0	17.21	Con h = 55 cm no cumple: Cortante 1D Y (D/C = 1.072, DCON5)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
9	29.31	100.0	17.04	Con h = 50 cm no cumple: Cortante 1D X (D/C = 1.174, DCON3); Cortante 1D Y (D/C = 1.213, DCON3)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
10	28.76	100.0	22.59	Con h = 60 cm no cumple: Cortante 1D X (D/C = 1.073, DCON3); Cortante 1D Y (D/C = 1.105, DCON3)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
11	31.46	100.0	13.07	Con h = 45 cm no cumple: Cortante 1D X (D/C = 1.112, DCON6); Cortante 1D Y (D/C = 1.226, DCON6)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
12	30.15	100.0	17.21	Con h = 55 cm no cumple: Cortante 1D Y (D/C = 1.030, DCON3)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)
13	31.77	100.0	13.07	Con h = 45 cm no cumple: Cortante 1D X (D/C = 1.125, DCON5); Cortante 1D Y (D/C = 1.240, DCON5)	CUMPLE	Barra recta	No requerido (contacto total en todas las combinaciones)

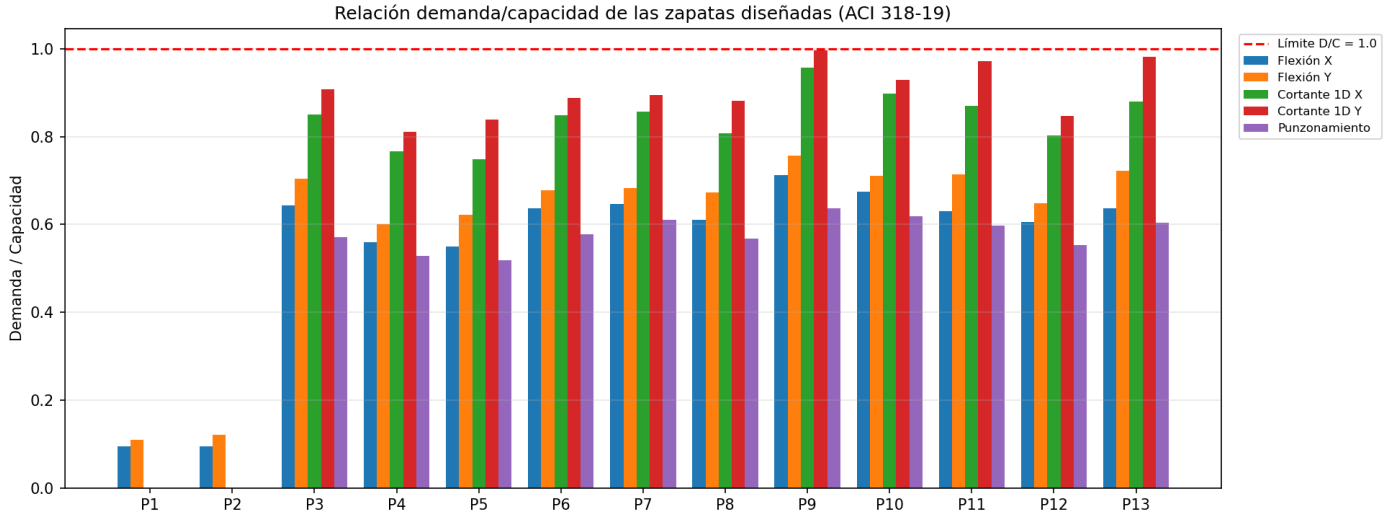


Figura B.1. Relación demanda/capacidad de cada verificación por zapata (salida del programa).

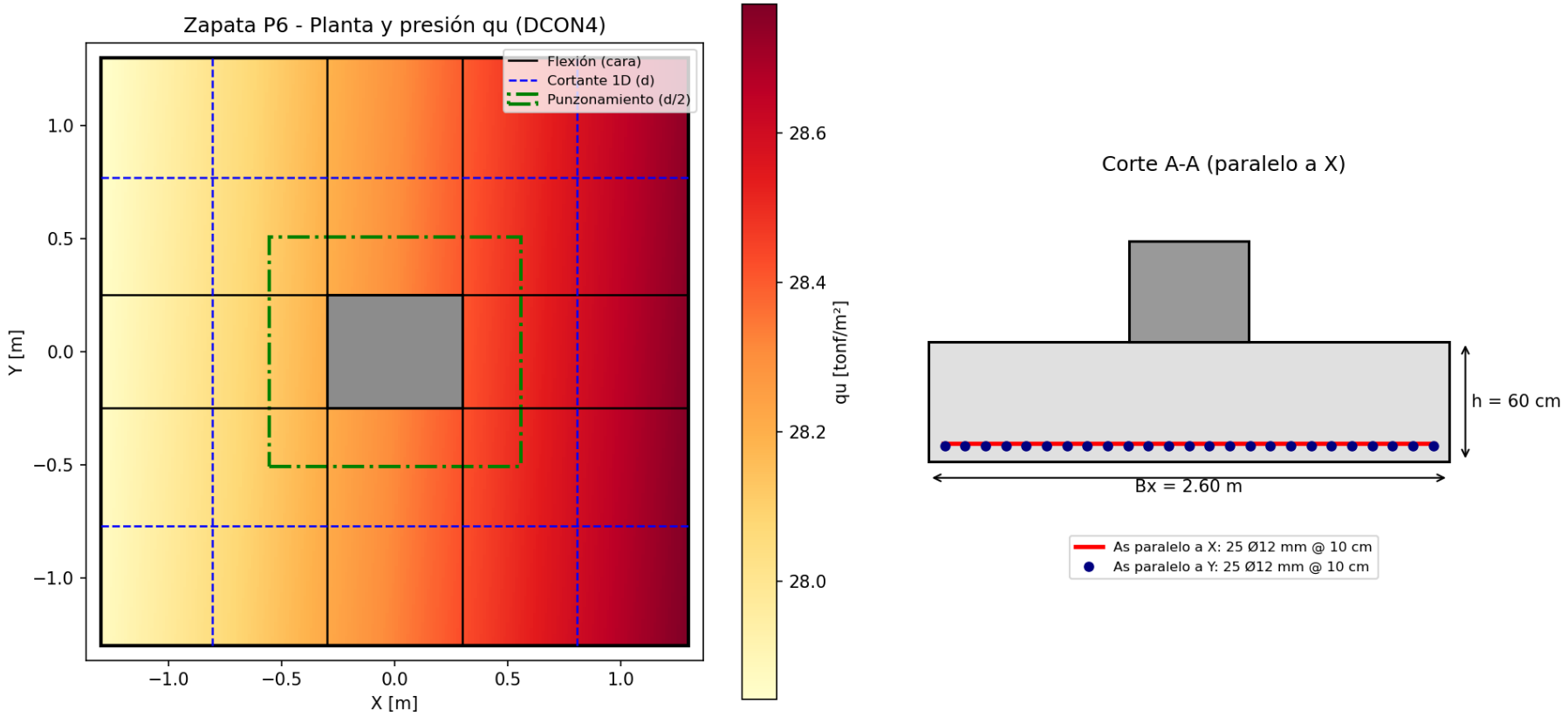


Figura B.2. Ejemplo de salida gráfica: zapata P6 (planta con armado, presiones de la combinación de punzonamiento y secciones críticas).

Anexo C. Script: Diseno_Zapatas_Aisladas.py

Código completo del programa (Python 3.10; numpy, pandas, matplotlib, openpyxl). Ejecución: `python Diseno_Zapatas_Aisladas.py`, *con InputData01.xlsx en la misma carpeta. Las salidas se escriben en la carpeta Resultados.*

```
# -*- coding: utf-8 -*-
"""
================================================================================
 DISEÑO ESTRUCTURAL DE ZAPATAS AISLADAS DE HORMIGÓN ARMADO  —  ACI 318-19
================================================================================
 Maestría en Construcción de Estructuras Sismorresistentes – UPS
 Asignatura : Diseño de Cimentaciones
 Tema       : Diseño de plintos aislados: flexión, cortante unidireccional y
              cortante bidireccional (punzonamiento).

 Entrada  : InputData01.xlsx, hoja "Tarea1". Reacciones en la base de las
            columnas (ETABS) para las combinaciones de diseño DCON1..DCON6,
            dimensiones de la zapata (Bx, By) y de la columna (Bcx, Bcy).
 Unidades : fuerzas [tonf], longitudes [m], presiones [tonf/m²],
            materiales [kg/cm²] (ecuaciones mks del ACI 318-19).
            1 kg/cm² = 10 tonf/m². Las verificaciones resistentes se evalúan en
            kgf-cm (Mu·10⁵ kgf·cm, b·100 cm) y se devuelven en tonf-m.

 Estructura del Excel (una fila por zapata y combinación; ver leer_datos):
   Point            identificación de la zapata
   Load             nombre de la combinación mayorada (DCON1, DCON2, ...)
   FZ [tonf]        carga axial mayorada; positiva = compresión sobre la zapata
   MX, MY [tonf·m]  momentos mayorados en la base de la columna
   FX, FY [tonf]    cortantes en la base (opcionales; solo con TRASLADAR_CORTANTE)
   Bx, By [m]       dimensiones en planta de la zapata (datos, no se modifican)
   Bcx, Bcy [m]     dimensiones de la columna
   ecx, ecy [m]     excentricidad del eje de la columna respecto al centro de la
                    zapata (opcionales; 0 si no existen)
   Se aceptan encabezados con la unidad entre corchetes, p. ej. "Bx[m]"; si la
   unidad no es la esperada el programa se detiene con un mensaje.

 Convención de signos (ejes con origen en el centro de la zapata):
   MY > 0 lleva la resultante hacia +X (ex = MY/P) y MX > 0 hacia +Y (ey = MX/P).
   Con columna centrada la zapata es simétrica y se revisa a ambos lados, así que
   el signo de MX y MY solo define qué borde se carga. Con columna excéntrica,
   ecx/ecy deben darse en este mismo sistema (el momento P·e se suma a MY/MX).

 Peso propio: las reacciones de ETABS no incluyen la zapata ni el relleno. Con
   INCLUIR_PESO_PROPIO = True se añade W = FACTOR_PESO·(γc·h + γs·(Df − h))·Bx·By,
   recalculado con cada h de la iteración. W aumenta la presión de contacto, pero
   también actúa hacia abajo sobre los voladizos, así que la flexión y el cortante
   se calculan con la presión neta qu − w (sin contarlo dos veces). Con contacto
   total W se anula y las solicitaciones son las mismas que con FZ sola.

 Criterio (parámetro CRITERIO): "ACI" es el diseño por defecto. "CLASE" reproduce las
   hipótesis del Excel de clase (d = h − rec, As,mín = 0.0018·b·d, barras a 15 cm,
   vc1 = 1.1·√f'c, sin transferencia de momento) y sirve para validar el script contra
   ese ejercicio; DIAMETRO_PUNTO permite imponer el diámetro elegido a mano en el Excel.

 Procedimiento para cada zapata:
   1. Presión de contacto última qu(x,y) para cada combinación, sin admitir
      tracciones en el suelo (contacto parcial resuelto numéricamente). Las
      combinaciones con contacto parcial se señalan (estado REVISAR) y las que
      quedan fuera del modelo (tracción o resultante fuera de la zapata) hacen
      que la zapata NO CUMPLA.
   2. Se propone un peralte h (desde H_MIN con incrementos DH) y un diámetro
      de barra.
   3. Flexión en la cara de la columna (ACI 13.2.7.1, 8.4.2.1) -> As, As,min.
   4. Cortante unidireccional a "d" de la cara (ACI 8.4.3.2, 22.5.5.1 c).
   5. Cortante bidireccional a "d/2" de la cara (ACI 22.6.4, 22.6.5.2, 8.4.4.2).
   6. Si alguna verificación no cumple se aumenta h y se repite.
   7. Verificaciones complementarias: anclaje (25.4), aplastamiento (22.8),
      zapatas rectangulares (13.3.3.3) y acero superior (18.13.2.5).

 Salidas  : carpeta "Resultados" con
            - Resultados_Zapatas.xlsx (resumen, detalle por combinación, parámetros)
            - Memoria_Calculo.txt     (memoria paso a paso de cada zapata)
            - Zapata_P#.png           (planta con presiones y secciones críticas)
            - Resumen_Demanda_Capacidad.png
================================================================================
"""

import os
import re
import math
import numpy as np
import pandas as pd
from matplotlib.figure import Figure
from matplotlib.patches import Rectangle

# ============================================================================
# 0. DATOS DE ENTRADA Y PARÁMETROS DE DISEÑO (modificables por el usuario)
# ============================================================================
CARPETA = os.path.dirname(os.path.abspath(__file__))
ARCHIVO_EXCEL = os.path.join(CARPETA, "InputData01.xlsx")
HOJA_EXCEL = "Tarea1"
CARPETA_SALIDA = os.path.join(CARPETA, "Resultados")

# --- Materiales ---------------------------------------------------------
FC = 210.0          # f'c del hormigón [kg/cm²]
FY = 4200.0         # fy del acero de refuerzo [kg/cm²]
LAMBDA = 1.0        # hormigón de peso normal (ACI 19.2.4)
LAMBDA_S = 1.0      # factor de tamaño; se permite despreciar en zapatas (ACI 13.2.6.2)
ALPHA_S = 40        # columna interior para punzonamiento (ACI 22.6.5.3)

# --- Factores de reducción de resistencia (ACI 21.2.1) ----------------------
```

```python
PHI_FLEXION = 0.90  # sección controlada por tracción
PHI_CORTE = 0.75
PHI_APLAST = 0.65

# --- Geometría y detallado ------------------------------------------------
REC = 0.075         # recubrimiento, hormigón contra el suelo [m] (ACI 20.5.1.3.1)
D_MIN = 0.15        # peralte efectivo mínimo [m] (ACI 13.3.1.2: 150 mm)
# Peralte inicial de búsqueda, igual para todas las zapatas. 40 cm es el espesor
# mínimo práctico adoptado; con él, P1 y P2 (1.10 m) tienen d ≥ voladizo y
# trabajan como hormigón en masa. Las demás zapatas necesitan más de 40 cm.
H_MIN = 0.40        # peralte inicial de prueba [m]
H_MAX = 1.50        # peralte máximo a probar [m]
DH = 0.05           # incremento del peralte [m]
# Opcional: peralte inicial distinto para zapatas específicas {Point: h [m]}.
# Vacío = todas empiezan en H_MIN (el script no depende de un archivo concreto).
H_MIN_PUNTO = {}
RHO_MIN = 0.0018    # cuantía mínima As,min = 0.0018·b·h (ACI 8.6.1.1, fy = 4200)
S_MIN = 0.10        # separación mínima práctica entre barras [m]
S_LIBRE_MIN = 0.025 # espaciamiento libre mínimo entre barras [m] (ACI 25.2.1)
# Diámetros en orden de preferencia [mm]. El Ø10 queda al final solo como
# recurso para anclar barras en voladizos muy cortos.
DIAMETROS = [12, 14, 16, 18, 20, 22, 25, 10]

# --- Peso propio de zapata y relleno --------------------------------------
GAMMA_C = 2.4       # peso específico del hormigón [tonf/m³]
GAMMA_S = 1.8       # peso específico del relleno [tonf/m³]
DF = 1.50           # profundidad de desplante [m]
# True: las cargas del Excel NO incluyen la zapata ni el relleno y se añaden con
# el h de cada iteración. False: las cargas ya los incluyen (no se suman de nuevo).
INCLUIR_PESO_PROPIO = True
# Factor de carga del peso propio. 0.9 (combinación 0.9D + E) es el valor
# conservador para el contacto con el suelo y para la flexión con contacto parcial.
FACTOR_PESO = 0.9
CONTACTO_TOTAL = 99.99  # [%] por debajo se considera pérdida de contacto parcial

# --- Opciones de cálculo --------------------------------------------------
# False: los cortantes FX, FY de la base los toman las cadenas / vigas de
#        amarre (criterio visto en clase), no generan momento en la zapata.
# True : se trasladan a la base de la zapata como momento adicional V·h.
TRASLADAR_CORTANTE = False
N_MALLA = 200       # divisiones por lado de la malla de integración de presiones

# --- Criterio de detallado y resistencia ----------------------------------
# "ACI"  : criterio del programa (por defecto). d descuenta el diámetro de cada
#          capa, As,mín = 0.0018·b·h, el armado cubre máx(As,req; As,mín) y el
#          punzonamiento usa vc = 1.06·√f'c con transferencia de momento γv·Msc.
# "CLASE": reproduce las hipótesis del Excel desarrollado en clase, para validar
#          el script: d = h − REC en ambas direcciones, As,mín = 0.0018·b·d,
#          barras a separación fija S_CLASE (n = B/s + 1), vc,1 = COEF_VC1_CLASE·√f'c
#          y Vu de punzonamiento sin transferencia de momento.
CRITERIO = "ACI"
S_CLASE = 0.15                      # separación fija de las barras [m]
COEF_VC1_CLASE = 1.1                # coeficiente de vc,1 en el Excel de clase
DIAMETROS_CLASE = [14, 16, 18, 20, 22, 25]   # diámetros [mm] en orden de preferencia
# Opcional: diámetro impuesto en zapatas específicas {Point: Ø [mm]}, en cualquier
# criterio (p. ej. para reproducir una elección manual del Excel). Vacío = automático.
DIAMETRO_PUNTO = {}


# ============================================================================
# 1. FUNCIONES AUXILIARES
# ============================================================================
MEMORIA = []


def log(texto=""):
    """Imprime en consola y guarda la línea para la memoria de cálculo."""
    print(texto)
    MEMORIA.append(texto)


def area_barra(db_mm):
    """Área de una barra [cm²] a partir de su diámetro [mm]."""
    return math.pi * (db_mm / 10.0) ** 2 / 4.0


def beta_1(fc):
    """Factor β1 del bloque de compresiones (ACI 22.2.2.4.3), f'c en kg/cm²."""
    if fc <= 280.0:
        return 0.85
    return max(0.65, 0.85 - 0.05 * (fc - 280.0) / 70.0)


class ErrorDatos(ValueError):
    """Error en los datos de entrada o en los parámetros, con un mensaje claro."""


# Columnas del Excel: nombre -> unidades aceptadas en el encabezado (entre corchetes)
COLUMNAS_OBLIGATORIAS = {"Point": None, "Load": None,
                         "FZ": ("tonf", "t", "ton"), "MX": ("tonf-m", "tonf·m", "t-m", "t·m", "tonf*m"),
                         "MY": ("tonf-m", "tonf·m", "t-m", "t·m", "tonf*m"),
                         "Bx": ("m",), "By": ("m",), "Bcx": ("m",), "Bcy": ("m",)}
COLUMNAS_OPCIONALES = {"FX": ("tonf", "t", "ton"), "FY": ("tonf", "t", "ton"),
                       "ecx": ("m",), "ecy": ("m",)}
COLUMNAS_NUMERICAS = ["FZ", "MX", "MY", "FX", "FY", "Bx", "By", "Bcx", "Bcy", "ecx", "ecy"]
COLUMNAS_GEOMETRIA = ["Bx", "By", "Bcx", "Bcy", "ecx", "ecy"]


def _filas_excel(indices, maximo=8):
    """Números de fila tal como se ven en Excel (encabezado en la fila 1)."""
    filas = [str(i + 2) for i in indices]
```

```
    return ", ".join(filas[:maximo]) + (" …" if len(filas) > maximo else "")


def texto_criterio():
    """Descripción corta del criterio de cálculo activo (memoria y Excel)."""
    if CRITERIO == "CLASE":
        return (f"CLASE (Excel de clase: d = h − rec, As,mín = 0.0018·b·d, barras @ "
                f"{S_CLASE * 100:.0f} cm, vc1 = {COEF_VC1_CLASE:g}·√f'c, sin transferencia de momento)")
    return ("ACI (d por capa, As,mín = 0.0018·b·h, vc1 = 1.06·√f'c, "
            "punzonamiento con transferencia de momento)")


def validar_parametros():
    """Revisa que los parámetros de la sección 0 tengan valores coherentes."""
    errores = []
    for nombre in ("FC", "FY", "REC", "H_MIN", "H_MAX", "DH", "S_MIN", "GAMMA_C", "DF"):
        valor = globals()[nombre]
        if not isinstance(valor, (int, float)) or not math.isfinite(valor) or valor <= 0:
            errores.append(f"{nombre} debe ser un número mayor que cero (valor: {valor}).")
    if not errores:
        if not 150 <= FC <= 700:
            errores.append(f"f'c = {FC} kg/cm² fuera del rango usual (150–700): ¿unidades?")
        if not 2000 <= FY <= 6000:
            errores.append(f"fy = {FY} kg/cm² fuera del rango usual (2000–6000): ¿unidades?")
        if REC >= 0.30:
            errores.append(f"Recubrimiento = {REC} m: debe darse en metros (p. ej. 0.075).")
        if H_MIN >= H_MAX:
            errores.append("El peralte inicial H_MIN debe ser menor que H_MAX.")
        if H_MAX > 5.0:
            errores.append(f"H_MAX = {H_MAX} m: debe darse en metros.")
    if not DIAMETROS or any(d <= 0 for d in DIAMETROS):
        errores.append("DIAMETROS debe contener al menos un diámetro positivo [mm].")
    if CRITERIO not in ("ACI", "CLASE"):
        errores.append(f'CRITERIO = "{CRITERIO}": debe ser "ACI" o "CLASE".')
    elif CRITERIO == "CLASE":
        if not 0.05 <= S_CLASE <= 0.45:
            errores.append(f"S_CLASE = {S_CLASE} m: separación fuera de 0.05–0.45 m.")
        if not DIAMETROS_CLASE or any(d <= 0 for d in DIAMETROS_CLASE):
            errores.append("DIAMETROS_CLASE debe contener al menos un diámetro positivo [mm].")
    if GAMMA_S < 0 or FACTOR_PESO < 0:
        errores.append("GAMMA_S y FACTOR_PESO no pueden ser negativos.")
    for punto, h in H_MIN_PUNTO.items():
        if not 0 < h < H_MAX:
            errores.append(f"H_MIN_PUNTO[{punto}] = {h} m debe estar entre 0 y H_MAX.")
    for punto, db in DIAMETRO_PUNTO.items():
        if not isinstance(db, (int, float)) or not 6 <= db <= 40:
            errores.append(f"DIAMETRO_PUNTO[{punto}] = {db}: diámetro en mm entre 6 y 40.")
    if errores:
        raise ErrorDatos("Parámetros no válidos:\n  - " + "\n  - ".join(errores))


def leer_datos(ruta, hoja):
    """Lee el Excel de entrada, normaliza los encabezados y valida los datos.
    Lanza ErrorDatos con un mensaje claro si falta algo o hay valores no válidos."""
    try:
        df = pd.read_excel(ruta, sheet_name=hoja)
    except FileNotFoundError:
        raise ErrorDatos(f"No se encontró el archivo de entrada: {ruta}")
    except ValueError as error:
        if "Worksheet" in str(error):
            hojas = pd.ExcelFile(ruta).sheet_names
            raise ErrorDatos(f"El Excel no tiene la hoja «{hoja}». Hojas disponibles: "
                             f"{', '.join(hojas)}.")
        raise ErrorDatos(f"No se pudo leer el Excel: {error}")

    # --- Encabezados: "Bx[m]" -> columna "Bx", unidad "m" ----------------------
    errores, nombres, unidades = [], {}, {}
    validas = {**COLUMNAS_OBLIGATORIAS, **COLUMNAS_OPCIONALES}
    for col in df.columns:
        m = re.match(r"^\s*([^\[\]]*?)\s*(?:\[\s*([^\]]*?)\s*\])?\s*$", str(col))
        base, unidad = (m.group(1), m.group(2)) if m else (str(col).strip(), None)
        clave = next((k for k in validas if k.lower() == base.lower()), base)
        nombres[col] = clave
        if clave in validas and unidad:
            unidades[clave] = unidad
    df = df.rename(columns=nombres)
    for clave, unidad in unidades.items():
        aceptadas = validas[clave]
        if aceptadas and unidad.lower().replace(" ", "") not in aceptadas:
            errores.append(f"Unidad inconsistente en «{clave}[{unidad}]»: se espera "
                           f"[{aceptadas[0]}].")
    faltan = [c for c in COLUMNAS_OBLIGATORIAS if c not in df.columns]
    if faltan:
        errores.append("Faltan las columnas obligatorias: " + ", ".join(faltan) +
                       ". Encabezados encontrados: " + ", ".join(map(str, df.columns)) + ".")
    if TRASLADAR_CORTANTE and not {"FX", "FY"} <= set(df.columns):
        errores.append("TRASLADAR_CORTANTE = True requiere las columnas FX y FY.")
    if errores:
        raise ErrorDatos("Datos de entrada no válidos:\n  - " + "\n  - ".join(errores))

    # --- Filas vacías y columnas opcionales ------------------------------------
    df = df.dropna(how="all").reset_index(drop=True)
    if df.empty:
        raise ErrorDatos(f"La hoja «{hoja}» no tiene filas de datos.")
    for col in COLUMNAS_OPCIONALES:
        if col not in df.columns:
            df[col] = 0.0

    # --- Valores numéricos y vacíos --------------------------------------------
    for col in ("Point", "Load"):
        vacias = df.index[df[col].isna()]
```

```
        if len(vacias):
            errores.append(f"«{col}» vacío en las filas {_filas_excel(vacias)}.")
    for col in COLUMNAS_NUMERICAS:
        numeros = pd.to_numeric(df[col], errors="coerce")
        malas = df.index[numeros.isna() & df[col].notna()]
        if len(malas):
            errores.append(f"«{col}» no es numérico en las filas {_filas_excel(malas)}.")
        vacias = df.index[df[col].isna()]
        if len(vacias) and col not in COLUMNAS_OPCIONALES:
            errores.append(f"«{col}» vacío en las filas {_filas_excel(vacias)}.")
        df[col] = numeros.fillna(0.0) if col in COLUMNAS_OPCIONALES else numeros
    if errores:
        raise ErrorDatos("Datos de entrada no válidos:\n  - " + "\n  - ".join(errores))
    df["Load"] = df["Load"].astype(str).str.strip()
    if pd.api.types.is_float_dtype(df["Point"]) and (df["Point"] % 1 == 0).all():
        df["Point"] = df["Point"].astype(int)

    # --- Geometría ----------------------------------------------------------
    for col in ("Bx", "By", "Bcx", "Bcy"):
        malas = df.index[df[col] <= 0]
        if len(malas):
            errores.append(f"«{col}» debe ser mayor que cero (filas {_filas_excel(malas)}).")
    grandes = df.index[(df[["Bx", "By"]] > 20).any(axis=1)]
    if len(grandes):
        errores.append(f"Bx o By mayor que 20 m en las filas {_filas_excel(grandes)}: "
                       "¿las dimensiones están en cm? Deben estar en metros.")
    for punto, grupo in df.groupby("Point", sort=False):
        distintas = [c for c in COLUMNAS_GEOMETRIA if grupo[c].nunique() > 1]
        if distintas:
            errores.append(f"Zapata {punto}: {', '.join(distintas)} cambian entre "
                           "combinaciones (deben ser iguales en todas sus filas).")
        f = grupo.iloc[0]
        if abs(f["ecx"]) + f["Bcx"] / 2 > f["Bx"] / 2 or abs(f["ecy"]) + f["Bcy"] / 2 > f["By"] / 2:
            errores.append(f"Zapata {punto}: la columna ({f['Bcx']:g} × {f['Bcy']:g} m, "
                           f"e = {f['ecx']:g}, {f['ecy']:g} m) sale de la zapata "
                           f"({f['Bx']:g} × {f['By']:g} m).")
        elif f["Bcx"] >= f["Bx"] or f["Bcy"] >= f["By"]:
            errores.append(f"Zapata {punto}: la columna no es menor que la zapata.")
        repetidas = grupo["Load"][grupo["Load"].duplicated()].unique()
        if len(repetidas):
            errores.append(f"Zapata {punto}: combinación repetida {', '.join(repetidas)}.")
    if errores:
        raise ErrorDatos("Datos de entrada no válidos:\n  - " + "\n  - ".join(errores))
    return df


# ============================================================================
# 2. PRESIÓN DE CONTACTO ÚLTIMA (sin tracciones en el suelo)
# ============================================================================
def presion_contacto(P, Mx, My, Bx, By, n=N_MALLA):
    """
    Distribución de presión qu(x,y) [tonf/m²] bajo la zapata.

    Se supone zapata rígida: la presión es un plano q = q0 + kx·x + ky·y.
    Si el plano elástico (P/A ± Mx·y/Ix ± My·x/Iy) tiene zonas negativas, se
    eliminan las tracciones y se busca por Newton-Raphson el plano que, con
    q = max(plano, 0), cumple el equilibrio:
        ΣF:  ∫q dA = P      ΣMy: ∫q·x dA = My      ΣMx: ∫q·y dA = Mx
    La zapata es simétrica, por lo que el signo de los momentos solo define
    hacia qué borde se cargan las presiones; las verificaciones se hacen a
    ambos lados de la columna.
    """
    x = (np.arange(n) + 0.5) * Bx / n - Bx / 2.0
    y = (np.arange(n) + 0.5) * By / n - By / 2.0
    X, Y = np.meshgrid(x, y)
    dA = (Bx / n) * (By / n)
    campo = dict(X=X, Y=Y, dA=dA, du=Bx / n, dv=By / n, P=P, Mx=Mx, My=My,
                 q=np.zeros_like(X), estado="OK", contacto=0.0, q_max=0.0,
                 q_min_elastico=np.nan, ex=np.nan, ey=np.nan)
    if P <= 0:
        campo["estado"] = "TRACCION"
        return campo

    ex, ey = My / P, Mx / P
    campo["ex"], campo["ey"] = ex, ey
    A = Bx * By
    Iy = By * Bx ** 3 / 12.0
    Ix = Bx * By ** 3 / 12.0
    campo["q_min_elastico"] = P / A - abs(Mx) * (By / 2) / Ix - abs(My) * (Bx / 2) / Iy
    if abs(ex) >= Bx / 2.0 or abs(ey) >= By / 2.0:
        campo["estado"] = "VOLCAMIENTO"      # resultante fuera de la zapata
        return campo

    objetivo = np.array([P, P * ex, P * ey])

    def residuo(p):
        plano = p[0] + p[1] * X + p[2] * Y
        q = np.maximum(plano, 0.0)
        R = np.array([q.sum(), (q * X).sum(), (q * Y).sum()]) * dA - objetivo
        return q, plano > 0.0, R

    p = np.array([P / A, P * ex / Iy, P * ey / Ix])   # solución elástica inicial
    q, activo, R = residuo(p)
    for _ in range(100):
        if np.linalg.norm(R) <= 1e-9 * P:
            break
        Xa, Ya = X[activo], Y[activo]
        J = np.array([[Xa.size, Xa.sum(), Ya.sum()],
                      [Xa.sum(), (Xa * Xa).sum(), (Xa * Ya).sum()],
                      [Ya.sum(), (Xa * Ya).sum(), (Ya * Ya).sum()]]) * dA
        dp = np.linalg.lstsq(J, -R, rcond=None)[0]
```

```
        t = 1.0
        while True:                                  # búsqueda lineal
            q_n, act_n, R_n = residuo(p + t * dp)
            if np.linalg.norm(R_n) < np.linalg.norm(R) or t < 1e-4:
                break
            t *= 0.5
        p = p + t * dp
        q, activo, R = q_n, act_n, R_n

    if np.linalg.norm(R) > 1e-6 * P:
        campo["estado"] = "NO CONVERGE"
    campo["q"] = q
    campo["plano"] = p
    campo["contacto"] = 100.0 * activo.mean()
    # presión máxima exacta: en la esquina más cargada
    campo["q_max"] = max(p[0] + abs(p[1]) * Bx / 2 + abs(p[2]) * By / 2, 0.0)
    return campo


def peso_cimiento(geo, h):
    """Peso mayorado de zapata + relleno: W [tonf] y carga repartida w [tonf/m²]."""
    if not INCLUIR_PESO_PROPIO:
        return 0.0, 0.0
    w = FACTOR_PESO * (GAMMA_C * h + GAMMA_S * max(DF - h, 0.0))
    return w * geo["Bx"] * geo["By"], w


def presiones_combinacion(c, geo, h):
    """Presión de contacto de una combinación para el peralte h.
    - Momentos en la base: MX, MY (+ V·h si TRASLADAR_CORTANTE).
    - Columna excéntrica: se suma P·e al momento respecto al centro de la zapata.
    - Peso propio W (si corresponde) centrado en la zapata.
    Devuelve el campo con q (presión total sobre el suelo) y q_net = q − w, que es
    la presión que flexiona y corta la zapata."""
    Mx_b, My_b = c["Mx"], c["My"]
    if TRASLADAR_CORTANTE:
        Mx_b, My_b = c["Mx"] - c["Vy"] * h, c["My"] + c["Vx"] * h
    W, w = peso_cimiento(geo, h)
    campo = presion_contacto(c["P"] + W, Mx_b + c["P"] * geo["ey"], My_b + c["P"] * geo["ex"],
                             geo["Bx"], geo["By"])
    campo.update(P=c["P"], Mx=Mx_b, My=My_b, W=W, w=w, q_net=campo["q"] - w,
                 Xc=campo["X"] - geo["ex"], Yc=campo["Y"] - geo["ey"])
    if campo["estado"] == "OK" and campo["contacto"] < CONTACTO_TOTAL:
        campo["estado"] = "CONTACTO PARCIAL"
    return campo


# Estados de presión que el modelo puede resolver (con contacto parcial se
# diseña, pero la zapata queda marcada para revisión)
ESTADOS_RESUELTOS = ("OK", "CONTACTO PARCIAL")


def texto_estado(cp):
    if cp["estado"] == "OK":
        return "Contacto total"
    if cp["estado"] == "CONTACTO PARCIAL":
        return f"CONTACTO PARCIAL ({cp['contacto']:.1f} %)"
    return {"TRACCION": "TRACCIÓN (P ≤ 0)",
            "VOLCAMIENTO": "RESULTANTE FUERA DE LA ZAPATA",
            "NO CONVERGE": "NO CONVERGE"}.get(cp["estado"], cp["estado"])


# ============================================================================
# 3. SOLICITACIONES EN LAS SECCIONES CRÍTICAS
# ============================================================================
def _fraccion_mas_alla(U, a, du, lado):
    """Fracción de cada celda (coordenada U, ancho du) situada más allá de
    la sección U·lado = a, y brazo de palanca de esa parte respecto a la sección."""
    w = np.clip((lado * U + du / 2.0 - a) / du, 0.0, 1.0)
    brazo = np.where(w >= 1.0, lado * U - a, w * du / 2.0)
    return w, brazo


def momentos_cara(campo, cx, cy):
    """Momento último en la cara de la columna (ACI 13.2.7.1, Tabla 13.2.7.1).
    Mu_x: momento que resisten las barras paralelas a X (voladizo en X, ancho By).
    Mu_y: momento que resisten las barras paralelas a Y (voladizo en Y, ancho Bx).
    Coordenadas medidas desde el eje de la columna y presión neta qu − w."""
    X, Y, q, dA = campo["Xc"], campo["Yc"], campo["q_net"], campo["dA"]
    Mu_x = Mu_y = 0.0
    for lado in (+1, -1):
        w, b = _fraccion_mas_alla(X, cx / 2.0, campo["du"], lado)
        Mu_x = max(Mu_x, float((q * w * b).sum() * dA))
        w, b = _fraccion_mas_alla(Y, cy / 2.0, campo["dv"], lado)
        Mu_y = max(Mu_y, float((q * w * b).sum() * dA))
    return Mu_x, Mu_y


def cortante_1D(campo, cx, cy, dx, dy):
    """Cortante unidireccional a una distancia d de la cara (ACI 8.4.3.2):
    resultante de la presión neta fuera de la sección crítica."""
    X, Y, q, dA = campo["Xc"], campo["Yc"], campo["q_net"], campo["dA"]
    Vu_x = Vu_y = 0.0
    for lado in (+1, -1):
        w, _ = _fraccion_mas_alla(X, cx / 2.0 + dx, campo["du"], lado)
        Vu_x = max(Vu_x, float((q * w).sum() * dA))
        w, _ = _fraccion_mas_alla(Y, cy / 2.0 + dy, campo["dv"], lado)
        Vu_y = max(Vu_y, float((q * w).sum() * dA))
    return Vu_x, Vu_y
```

```
def punzonamiento(campo, cx, cy, d):
    """Cortante bidireccional en el perímetro crítico a d/2 de las caras
    (ACI 22.6.4.1) con transferencia de momento por excentricidad del
    cortante (ACI 8.4.4.2). Devuelve el esfuerzo máximo vu [tonf/m²].
    Vu = P − ∫(qu − w) dentro del perímetro (el peso propio no se cuenta dos veces)."""
    X, Y, q, dA = campo["Xc"], campo["Yc"], campo["q_net"], campo["dA"]
    b1, b2 = cx + d, cy + d                  # lados del perímetro crítico
    bo = 2.0 * (b1 + b2)
    wx = np.clip((b1 / 2.0 + campo["du"] / 2.0 - np.abs(X)) / campo["du"], 0, 1)
    wy = np.clip((b2 / 2.0 + campo["dv"] / 2.0 - np.abs(Y)) / campo["dv"], 0, 1)
    dentro = wx * wy
    R_dentro = float((q * dentro).sum() * dA)
    Vu = campo["P"] - R_dentro               # reacción del suelo fuera del perímetro
    # Momento desbalanceado que debe transferir la sección crítica
    Msc_y = abs(float((q * X * (1 - dentro)).sum() * dA))   # alrededor de Y
    Msc_x = abs(float((q * Y * (1 - dentro)).sum() * dA))   # alrededor de X
    # Fracciones de momento transferidas por excentricidad del cortante
    gv_y = 1.0 - 1.0 / (1.0 + (2.0 / 3.0) * math.sqrt(b1 / b2))
    gv_x = 1.0 - 1.0 / (1.0 + (2.0 / 3.0) * math.sqrt(b2 / b1))
    # Propiedad Jc de la sección crítica (columna interior, R8.4.4.2.3)
    Jc_y = d * b1 ** 3 / 6.0 + b1 * d ** 3 / 6.0 + d * b2 * b1 ** 2 / 2.0
    Jc_x = d * b2 ** 3 / 6.0 + b2 * d ** 3 / 6.0 + d * b1 * b2 ** 2 / 2.0
    vu = Vu / (bo * d)
    if CRITERIO == "ACI":   # el Excel de clase no incluye la transferencia de momento
        vu += gv_y * Msc_y * (b1 / 2.0) / Jc_y + gv_x * Msc_x * (b2 / 2.0) / Jc_x
    return dict(Vu=Vu, vu=vu, bo=bo, b1=b1, b2=b2, Msc_x=Msc_x, Msc_y=Msc_y,
                gv_x=gv_x, gv_y=gv_y)


# ============================================================================
# 4. RESISTENCIAS DE DISEÑO (ACI 318-19, unidades mks)
# ============================================================================
def acero_flexion(Mu, b, d):
    """As requerido [cm²] para Mu [tonf·m] en una sección b x d [m].
    Mu = ϕ·As·fy·(d − a/2), a = As·fy/(0.85·f'c·b). None si no alcanza."""
    Mu_kgcm = Mu * 1e5
    b_cm, d_cm = b * 100.0, d * 100.0
    Rn = Mu_kgcm / (PHI_FLEXION * b_cm * d_cm ** 2)
    disc = 1.0 - 2.0 * Rn / (0.85 * FC)
    if disc < 0:
        return None
    rho = 0.85 * FC / FY * (1.0 - math.sqrt(disc))
    return rho * b_cm * d_cm


def momento_resistente(As, b, d):
    """ϕMn [tonf·m] de una sección rectangular con As [cm²]. ϕ según εt (21.2.2)."""
    b_cm, d_cm = b * 100.0, d * 100.0
    a = As * FY / (0.85 * FC * b_cm)
    c = a / beta_1(FC)
    eps_t = 0.003 * (d_cm - c) / c
    eps_y = FY / 2.0e6
    if eps_t >= eps_y + 0.003:
        phi = 0.90
    elif eps_t <= eps_y:
        phi = 0.65
    else:
        phi = 0.65 + 0.25 * (eps_t - eps_y) / 0.003
    return phi * As * FY * (d_cm - a / 2.0) / 1e5, a, eps_t, phi


def resistencia_cortante_1D(As, b, d):
    """ϕVc [tonf] para cortante unidireccional sin refuerzo transversal,
    ACI 22.5.5.1(c): Vc = 2.12·λs·λ·(ρw)^(1/3)·√f'c·bw·d (Nu = 0),
    con el límite Vc ≤ 1.33·λ·√f'c·bw·d (22.5.5.1.1)."""
    rho_w = As / (b * 100.0 * d * 100.0)
    vc = 2.12 * LAMBDA_S * LAMBDA * rho_w ** (1.0 / 3.0) * math.sqrt(FC)
    vc = min(vc, 1.33 * LAMBDA * math.sqrt(FC))          # kg/cm²
    return PHI_CORTE * vc * 10.0 * b * d, vc, rho_w


def resistencia_punzonamiento(cx, cy, d, bo):
    """ϕvc [tonf/m²] para cortante bidireccional, ACI Tabla 22.6.5.2 (mks)."""
    beta = max(cx, cy) / min(cx, cy)
    vc = min(1.06 if CRITERIO == "ACI" else COEF_VC1_CLASE,
             0.53 * (1.0 + 2.0 / beta),
             0.27 * (2.0 + ALPHA_S * d / bo)) * LAMBDA_S * LAMBDA * math.sqrt(FC)
    return PHI_CORTE * vc * 10.0, vc


def longitud_desarrollo(db_mm):
    """ld (barra recta, ACI 25.4.2.3) y ldh (gancho estándar, ACI 25.4.3.1) [m].
    ψt = ψe = ψg = 1 (barra inferior, sin recubrimiento epóxico, grado 420)."""
    db = db_mm / 10.0                                    # cm
    k = 6.6 if db_mm <= 19 else 5.3
    ld = max(FY / (k * LAMBDA * math.sqrt(FC)) * db, 30.0)
    fc_mpa, fy_mpa = FC * 0.0980665, FY * 0.0980665
    psi_c = fc_mpa / 105.0 + 0.6 if fc_mpa < 40 else 1.0
    psi_r = 1.6                                          # sin confinamiento
    ldh = fy_mpa * psi_r * psi_c / (23.0 * LAMBDA * math.sqrt(fc_mpa)) * db_mm ** 1.5
    ldh = max(ldh / 10.0, 8.0 * db, 15.0)
    return ld / 100.0, ldh / 100.0


def voladizos(geo):
    """Voladizos medidos desde las caras de la columna [m]: (menor X, mayor X,
    menor Y, mayor Y). Iguales a ambos lados si la columna está centrada."""
    vx = (geo["Bx"] - geo["cx"]) / 2.0
    vy = (geo["By"] - geo["cy"]) / 2.0
    return (vx - abs(geo["ex"]), vx + abs(geo["ex"]), vy - abs(geo["ey"]), vy + abs(geo["ey"]))
```

```python


def aplastamiento(Pu, geo, h):
    """Resistencia al aplastamiento en el contacto columna-zapata (ACI 22.8.3.2)."""
    cx, cy = geo["cx"], geo["cy"]
    A1 = cx * cy
    # √(A2/A1): A2 concéntrica y semejante a A1, limitada por la pirámide 1:2 y
    # por el borde más cercano de la zapata
    vx_min, _, vy_min, _ = voladizos(geo)
    k = min((cx + 2 * min(vx_min, 2 * h)) / cx, (cy + 2 * min(vy_min, 2 * h)) / cy, 2.0)
    phiBn_zap = PHI_APLAST * 0.85 * FC * 10.0 * A1 * k
    phiBn_col = PHI_APLAST * 0.85 * FC * 10.0 * A1
    return min(phiBn_zap, phiBn_col)


def distribuir_barras(As, ancho, db_mm, h):
    """Número y separación de barras para cubrir As [cm²] en un ancho [m].
    Separación máxima min(2h, 45 cm) en secciones críticas (ACI 8.7.2.2)."""
    Ab = area_barra(db_mm)
    s_max = min(2.0 * h, 0.45)
    L = ancho - 2.0 * REC - db_mm / 1000.0
    n = max(math.ceil(As / Ab), math.ceil(L / s_max) + 1, 2)
    if CRITERIO == "CLASE":
        # Excel de clase: número de barras fijado por la separación, n = B/s + 1
        n = max(n, math.floor(ancho / S_CLASE + 1e-9) + 1)
    s = L / (n - 1)
    return dict(n=n, db=db_mm, s=s, As=n * Ab, s_max=s_max)


def texto_armado(barras):
    return f"{barras['n']} Ø{barras['db']} mm @ {math.floor(barras['s'] * 100)} cm"


def distribucion_rectangular(As, Bx, By, db_mm, h):
    """ACI 13.3.3.3: en zapatas rectangulares el acero de la dirección corta se
    concentra γs·As en una banda central de ancho igual al lado corto."""
    if abs(Bx - By) < 1e-6:
        return "No aplica (zapata cuadrada: distribución uniforme, ACI 13.3.3.2)"
    L, B = max(Bx, By), min(Bx, By)
    gs = 2.0 / (L / B + 1.0)
    banda = distribuir_barras(gs * As, B + 2 * REC, db_mm, h)
    fuera = distribuir_barras((1 - gs) * As / 2.0, (L - B) / 2.0 + 2 * REC, db_mm, h)
    return (f"γs = {gs:.3f}: banda central {B:.2f} m -> {texto_armado(banda)}; "
            f"a cada lado -> {texto_armado(fuera)}")


# ============================================================================
# 5. DISEÑO DE UNA ZAPATA
# ============================================================================
def verificar(geo, combos, campos, h, db_mm):
    """Verifica la zapata para un peralte h y un diámetro de barra dados."""
    Bx, By, cx, cy = geo["Bx"], geo["By"], geo["cx"], geo["cy"]
    dbm = db_mm / 1000.0
    validos = [i for i, c in enumerate(campos) if c["estado"] in ESTADOS_RESUELTOS]

    # --- Flexión: momentos en la cara (independientes de d) -----------------
    Mx_list = [momentos_cara(campos[i], cx, cy) for i in validos]
    Mu_x = max(m[0] for m in Mx_list)
    Mu_y = max(m[1] for m in Mx_list)

    # Peralte efectivo = h − recubrimiento − medio diámetro de la barra de la
    # capa analizada. Las barras de una dirección se apoyan sobre las de la otra,
    # así que la capa superior pierde un diámetro: d = h − rec − 1.5·db.
    # La dirección con mayor momento va en la capa inferior (mayor d) y cada
    # dirección se revisa con su propio d, que en la capa superior es el más
    # desfavorable.
    # En el criterio CLASE se reproduce el Excel: d = h − REC en ambas direcciones.
    if CRITERIO == "CLASE":
        dx = dy = h - REC
    elif Mu_x >= Mu_y:
        dx, dy = h - REC - dbm / 2.0, h - REC - 1.5 * dbm
    else:
        dx, dy = h - REC - 1.5 * dbm, h - REC - dbm / 2.0
    dm = (dx + dy) / 2.0                          # d promedio (ACI 22.6.2.1)
    r = dict(h=h, db=db_mm, dx=dx, dy=dy, d=dm, Mu_x=Mu_x, Mu_y=Mu_y,
             cumple=False, motivo="")
    if min(dx, dy) < D_MIN:
        r["motivo"] = "d < 15 cm (ACI 13.3.1.2)"
        return r
    # Hormigón en masa: si d ≥ voladizo, la sección crítica de cortante (a d de
    # la cara) queda fuera de la zapata en esa dirección (se usa el voladizo mayor).
    _, vx_max, _, vy_max = voladizos(geo)
    masa_x = dx >= vx_max
    masa_y = dy >= vy_max
    bloque = masa_x and masa_y
    # El perímetro de punzonamiento a d/2 debe quedar completo dentro de la zapata
    # (modelo de columna interior); si lo corta un borde, el modelo no aplica.
    perimetro_ok = ((cx + dm) / 2.0 + abs(geo["ex"]) <= Bx / 2.0 + 1e-9 and
                    (cy + dm) / 2.0 + abs(geo["ey"]) <= By / 2.0 + 1e-9)
    r.update(masa_x=masa_x, masa_y=masa_y, bloque=bloque, perimetro_ok=perimetro_ok)

    # ACI 8.6.1.1: 0.0018·Ag (b·h). El Excel de clase la aplica sobre b·d.
    As_min_x = RHO_MIN * By * 100 * (h if CRITERIO == "ACI" else dx) * 100
    As_min_y = RHO_MIN * Bx * 100 * (h if CRITERIO == "ACI" else dy) * 100
    As_req_x = acero_flexion(Mu_x, By, dx)
    As_req_y = acero_flexion(Mu_y, Bx, dy)
    if As_req_x is None or As_req_y is None:
        r["motivo"] = "Sección insuficiente a flexión"
        return r
    bar_x = distribuir_barras(max(As_req_x, As_min_x), By, db_mm, h)
    bar_y = distribuir_barras(max(As_req_y, As_min_y), Bx, db_mm, h)
```

```
    phiMn_x, a_x, et_x, phi_x = momento_resistente(bar_x["As"], By, dx)
    phiMn_y, a_y, et_y, phi_y = momento_resistente(bar_y["As"], Bx, dy)
    r.update(As_req_x=As_req_x, As_req_y=As_req_y, As_min_x=As_min_x,
             As_min_y=As_min_y, bar_x=bar_x, bar_y=bar_y,
             phiMn_x=phiMn_x, phiMn_y=phiMn_y, eps_t_x=et_x, eps_t_y=et_y,
             phi_x=phi_x, phi_y=phi_y,
             DC_flex_x=Mu_x / phiMn_x, DC_flex_y=Mu_y / phiMn_y)

    # --- Resistencias a cortante -------------------------------------------
    phiVc_x, vc_x, rho_x = resistencia_cortante_1D(bar_x["As"], By, dx)
    phiVc_y, vc_y, rho_y = resistencia_cortante_1D(bar_y["As"], Bx, dy)
    b1, b2 = cx + dm, cy + dm
    phi_vc_p, vc_p = resistencia_punzonamiento(cx, cy, dm, 2 * (b1 + b2))
    r.update(phiVc_x=phiVc_x, phiVc_y=phiVc_y, vc_x=vc_x, vc_y=vc_y,
             rho_w_x=rho_x, rho_w_y=rho_y, phi_vc_p=phi_vc_p, vc_p=vc_p)

    # --- Revisión combinación por combinación ------------------------------
    detalle = []
    for k, i in enumerate(validos):
        c, campo = combos[i], campos[i]
        Vu_x, Vu_y = cortante_1D(campo, cx, cy, dx, dy)
        pz = punzonamiento(campo, cx, cy, dm)
        detalle.append(dict(
            Combo=c["nombre"], Pu=c["P"], Mux_base=campo["Mx"], Muy_base=campo["My"],
            ex=campo["ex"], ey=campo["ey"], qu_max=campo["q_max"],
            qu_min_elastico=campo["q_min_elastico"], contacto=campo["contacto"],
            Mu_x=Mx_list[k][0], Mu_y=Mx_list[k][1],
            Vu1_x=Vu_x, Vu1_y=Vu_y, Vu_p=pz["Vu"], Msc_x=pz["Msc_x"],
            Msc_y=pz["Msc_y"], vu_p=pz["vu"],
            DC_flex_x=Mx_list[k][0] / phiMn_x, DC_flex_y=Mx_list[k][1] / phiMn_y,
            DC_v1_x=Vu_x / phiVc_x, DC_v1_y=Vu_y / phiVc_y,
            DC_punz=pz["vu"] / phi_vc_p, idx=i, bo=pz["bo"]))
    r["detalle"] = detalle

    # Caso gobernante de cada verificación (se conservan sus solicitaciones)
    for clave, extra in (("DC_v1_x", ("Vu1_x",)), ("DC_v1_y", ("Vu1_y",)),
                         ("DC_punz", ("Vu_p", "vu_p"))):
        gob = max(detalle, key=lambda e: e[clave])
        r[clave] = gob[clave]
        r[clave + "_combo"] = gob["Combo"]
        for k in extra:
            r[k] = gob[k]
    r["combo_flex_x"] = max(detalle, key=lambda e: e["Mu_x"])["Combo"]
    r["combo_flex_y"] = max(detalle, key=lambda e: e["Mu_y"])["Combo"]
    r["idx_punz"] = max(detalle, key=lambda e: e["DC_punz"])["idx"]
    # Si la sección a d de la cara queda fuera del voladizo no hay cortante en
    # esa dirección: el voladizo trabaja como hormigón en masa.
    claves = []
    if masa_x:
        claves.append("DC_v1_x")
    if masa_y:
        claves.append("DC_v1_y")
    if bloque:
        # En ambas direcciones: la falla a 45° sale por los bordes, tampoco hay punzonamiento
        claves.append("DC_punz")
        r["idx_punz"] = max(detalle, key=lambda e: e["qu_max"])["idx"]
    for clave in claves:
        r[clave] = float("nan")
        r[clave + "_combo"] = "No aplica"

    # --- Verificación con la mayor relación D/C (la que gobierna) ----------
    candidatos = [("Flexión X", r["DC_flex_x"], r["combo_flex_x"]),
                  ("Flexión Y", r["DC_flex_y"], r["combo_flex_y"]),
                  ("Cortante 1D X", r["DC_v1_x"], r["DC_v1_x_combo"]),
                  ("Cortante 1D Y", r["DC_v1_y"], r["DC_v1_y_combo"]),
                  ("Punzonamiento", r["DC_punz"], r["DC_punz_combo"])]
    candidatos = [c for c in candidatos if not math.isnan(c[1])]
    r["gobierna"] = max(candidatos, key=lambda c: c[1])

    # --- Criterio de aceptación por resistencia ----------------------------
    fallas = [f"{nombre} (D/C = {dc:.3f}, {combo})"
              for nombre, dc, combo in candidatos if dc > 1.0]
    if not bloque and not perimetro_ok:
        fallas.append("el perímetro de punzonamiento sale de la zapata (columna de borde: "
                      "fuera del alcance del modelo de columna interior)")
    if min(bar_x["s"], bar_y["s"]) - dbm < S_LIBRE_MIN:
        fallas.append("espaciamiento libre < 2.5 cm (ACI 25.2.1)")
    r["cumple"] = not fallas
    r["motivo"] = "; ".join(fallas)
    return r


def elegir_diametro_clase(geo, combos, campos, h, diametros):
    """Criterio CLASE (Excel de clase): barras a separación fija S_CLASE; se toma el
    primer diámetro cuyo número de barras n = B/s + 1 cubre máx(As,req; As,mín). Si
    ninguno lo cubre, el último probado (con más barras). Si el resultado no cumple
    cortante o punzonamiento, el peralte aumenta."""
    n_fijo_x = math.floor(geo["By"] / S_CLASE + 1e-9) + 1
    n_fijo_y = math.floor(geo["Bx"] / S_CLASE + 1e-9) + 1
    r = None
    for db_mm in diametros:
        r = verificar(geo, combos, campos, h, db_mm)
        if "bar_x" not in r:
            continue
        r["anclaje"] = revisar_anclaje(geo, r)
        if r["bar_x"]["n"] == n_fijo_x and r["bar_y"]["n"] == n_fijo_y:
            return r
    return r


def disenar_zapata(punto, datos):
```

```
    """Itera peralte h y diámetro de barra hasta cumplir todas las verificaciones."""
    f0 = datos.iloc[0]
    geo = dict(Bx=float(f0["Bx"]), By=float(f0["By"]),
               cx=float(f0["Bcx"]), cy=float(f0["Bcy"]),
               ex=float(f0["ecx"]), ey=float(f0["ecy"]))
    combos = [dict(nombre=f["Load"], P=float(f["FZ"]), Mx=float(f["MX"]),
                   My=float(f["MY"]), Vx=float(f["FX"]), Vy=float(f["FY"]))
              for _, f in datos.iterrows()]

    campos = None
    ultimo = None
    h_inicial = H_MIN_PUNTO.get(punto, H_MIN)
    if punto in DIAMETRO_PUNTO:
        diametros = [DIAMETRO_PUNTO[punto]]
    else:
        diametros = DIAMETROS_CLASE if CRITERIO == "CLASE" else DIAMETROS
    historial = []                          # (h, motivo) de los peraltes que no cumplen
    for h in np.arange(h_inicial, H_MAX + 1e-9, DH):
        h = round(float(h), 3)
        # Presiones: dependen de h si se suma el peso propio o se traslada el
        # cortante; si no, se calculan una sola vez.
        if campos is None or TRASLADAR_CORTANTE or INCLUIR_PESO_PROPIO:
            campos = [presiones_combinacion(c, geo, h) for c in combos]
        if not any(cp["estado"] in ESTADOS_RESUELTOS for cp in campos):
            break
        # Diámetro: el primero de la lista con separación ≥ S_MIN que además se
        # ancla dentro del voladizo. Si ninguno se ancla así, se acepta el primero
        # que se ancla con el espaciamiento libre mínimo del ACI (25.2.1); y si
        # tampoco, el primero con s ≥ S_MIN (el anclaje queda reportado).
        if CRITERIO == "CLASE":
            r = elegir_diametro_clase(geo, combos, campos, h, diametros)
        else:
            elegido = respaldo = ajustado = None
            for db_mm in diametros:
                r = verificar(geo, combos, campos, h, db_mm)
                if "bar_x" not in r:
                    continue
                s = min(r["bar_x"]["s"], r["bar_y"]["s"])
                r["anclaje"] = revisar_anclaje(geo, r)
                if s >= S_MIN:
                    respaldo = respaldo or r
                    if r["anclaje"]["cumple"]:
                        elegido = r
                        break
                elif ajustado is None and r["anclaje"]["cumple"]:
                    ajustado = r
            r = elegido or ajustado or respaldo or r
        ultimo = (r, campos)
        # Con el armado elegido, si no cumple resistencia se aumenta el peralte
        if r["cumple"]:
            break
        historial.append((h, r["motivo"]))

    if ultimo is None:
        # Ninguna combinación tiene una presión de contacto resoluble
        r = dict(h=h_inicial, cumple=False, bloque=False,
                 motivo="ninguna combinación mantiene la resultante dentro de la zapata")
        campos = [presiones_combinacion(c, geo, h_inicial) for c in combos]
        ultimo = (r, campos)
    r, campos = ultimo
    if "anclaje" not in r and "bar_x" in r:
        r["anclaje"] = revisar_anclaje(geo, r)
    r["geo"], r["combos"], r["campos"], r["punto"] = geo, combos, campos, punto
    r["estado_combos"] = {c["nombre"]: cp["estado"] for c, cp in zip(combos, campos)}
    r["resiste"] = r["cumple"]

    # --- Verificación que controla el peralte final --------------------------
    if r["resiste"]:
        if historial:
            h_ant, motivo = historial[-1]
            r["control_h"] = f"Con h = {h_ant * 100:.0f} cm no cumple: {motivo}"
        else:
            r["control_h"] = (f"Peralte inicial de búsqueda ({h_inicial * 100:.0f} cm): "
                              "cumple todas las verificaciones con el primer h probado")
    else:
        r["control_h"] = (f"Sin solución entre h = {h_inicial * 100:.0f} y "
                          f"{H_MAX * 100:.0f} cm")
        r["motivo"] = f"{r['control_h']}: {r['motivo']}"

    # --- Control de las combinaciones: pérdida de contacto -------------------
    fuera = [f"{c['nombre']}: {texto_estado(cp)}" for c, cp in zip(combos, campos)
             if cp["estado"] not in ESTADOS_RESUELTOS]
    parcial = [f"{c['nombre']} ({cp['contacto']:.1f} % en contacto)"
               for c, cp in zip(combos, campos) if cp["estado"] == "CONTACTO PARCIAL"]
    r["advertencias"] = []
    if fuera:
        r["advertencias"].append("Combinaciones fuera del modelo de presiones (no se "
                                 "verifican): " + "; ".join(fuera))
    if parcial:
        r["advertencias"].append("Pérdida de contacto parcial con el suelo en " +
                                 ", ".join(parcial) + ". Revisar estabilidad y presión "
                                 "admisible; Bx, By son datos y no se modifican")
    if not r["resiste"]:
        r["estado"] = "NO CUMPLE"
    elif fuera:
        r["estado"] = "NO CUMPLE"
        r["motivo"] = "combinaciones fuera del modelo de presiones: " + "; ".join(fuera)
    elif parcial:
        r["estado"] = "REVISAR"
        r["motivo"] = "pérdida de contacto parcial en " + ", ".join(parcial)
    else:
```

```
        r["estado"] = "CUMPLE"
    r["cumple"] = r["estado"] == "CUMPLE"

    # --- Aplastamiento columna-zapata ----------------------------------------
    Pu_max = max(c["P"] for c in combos)
    r["Pu_max"] = Pu_max
    r["phiBn"] = aplastamiento(Pu_max, geo, r["h"])

    # --- Acero superior si hay levantamiento (ACI 18.13.2.5) ----------------
    levanta = any(cp["estado"] != "OK" for cp in campos)
    r["levantamiento"] = levanta
    r["acero_superior"] = "No requerido (contacto total en todas las combinaciones)"
    if levanta and "bar_x" in r:
        h = r["h"]
        w = 1.2 * (GAMMA_C * h + GAMMA_S * max(DF - h, 0.0))   # peso propio + relleno
        _, Lx, _, Ly = voladizos(geo)
        d_sup = h - REC - r["db"] / 1000.0
        As_tx = max(acero_flexion(w * geo["By"] * Lx ** 2 / 2.0, geo["By"], d_sup) or 0.0,
                    RHO_MIN * geo["By"] * 100 * h * 100)
        As_ty = max(acero_flexion(w * geo["Bx"] * Ly ** 2 / 2.0, geo["Bx"], d_sup) or 0.0,
                    RHO_MIN * geo["Bx"] * 100 * h * 100)
        r["acero_superior"] = (
            f"Paralelo a X: {texto_armado(distribuir_barras(As_tx, geo['By'], r['db'], h))}; "
            f"paralelo a Y: {texto_armado(distribuir_barras(As_ty, geo['Bx'], r['db'], h))}")
    if "bar_x" in r:
        As_ref = max(r["As_req_x"], r["As_min_x"])
        r["rectangular"] = distribucion_rectangular(As_ref, geo["Bx"], geo["By"], r["db"], r["h"])
    return r


def revisar_anclaje(geo, r):
    """Longitud disponible desde la cara de la columna hasta el extremo de la
    barra vs. ld recta o ldh con gancho estándar de 90° (ACI 13.2.8, 25.4)."""
    ld, ldh = longitud_desarrollo(r["db"])
    vx_min, _, vy_min, _ = voladizos(geo)
    disp = min(vx_min, vy_min) - REC          # voladizo más corto
    if ld <= disp:
        tipo, ok = "Barra recta", True
    elif ldh <= disp:
        tipo, ok = "Gancho estándar 90°", True
    else:
        tipo, ok = "NO CUMPLE (ampliar zapata o reducir Ø)", False
    return dict(ld=ld, ldh=ldh, disp=disp, tipo=tipo, cumple=ok)


# ============================================================================
# 6. MEMORIA DE CÁLCULO
# ============================================================================
def imprimir_zapata(r):
    g = r["geo"]
    log("=" * 79)
    log(f" ZAPATA PUNTO {r['punto']}   Bx = {g['Bx']:.2f} m   By = {g['By']:.2f} m   "
        f"Columna {g['cx']*100:.0f} x {g['cy']*100:.0f} cm")
    if g["ex"] or g["ey"]:
        log(f" Columna excéntrica: ecx = {g['ex']*100:.1f} cm, ecy = {g['ey']*100:.1f} cm")
    log("=" * 79)
    if "bar_x" not in r:
        log(f" *** No fue posible diseñar: {r['motivo']}")
        for a in r.get("advertencias", []):
            log(f" ADVERTENCIA: {a}")
        return
    log(f" Peralte adoptado h = {r['h']*100:.0f} cm,  Ø = {r['db']} mm")
    log(f" Verificación que controla h: {r['control_h']}")
    db = r["db"] / 10.0
    if CRITERIO == "CLASE":
        log(f" dx = dy = h − rec = {r['h']*100:.0f} − {REC*100:.1f} = {r['dx']*100:.1f} cm "
            "(criterio CLASE, igual que el Excel de clase)")
    else:
        capa_x = "inferior" if r["dx"] >= r["dy"] else "superior"
        capa_y = "superior" if capa_x == "inferior" else "inferior"
        log(f" dx = h − rec − {'0.5' if capa_x == 'inferior' else '1.5'}·db = {r['h']*100:.0f} − "
            f"{REC*100:.1f} − {(0.5 if capa_x == 'inferior' else 1.5)*db:.1f} = {r['dx']*100:.1f} cm "
            f"(barras paralelas a X, capa {capa_x})")
        log(f" dy = h − rec − {'0.5' if capa_y == 'inferior' else '1.5'}·db = {r['h']*100:.0f} − "
            f"{REC*100:.1f} − {(0.5 if capa_y == 'inferior' else 1.5)*db:.1f} = {r['dy']*100:.1f} cm "
            f"(barras paralelas a Y, capa {capa_y})")
    log(f" d promedio = {r['d']*100:.1f} cm   (d ≥ 15 cm ✓)")
    log("")
    log(" 1) PRESIONES DE CONTACTO ÚLTIMAS")
    cp0 = r["campos"][0]
    if INCLUIR_PESO_PROPIO:
        log(f"   Peso propio zapata + relleno: w = {FACTOR_PESO:g}·({GAMMA_C:g}·{r['h']:.2f} + "
            f"{GAMMA_S:g}·{max(DF - r['h'], 0):.2f}) = {cp0['w']:.3f} t/m²,  "
            f"W = w·Bx·By = {cp0['W']:.2f} t")
        log("   qmax incluye W; flexión y cortante usan la presión neta qu − w.")
    else:
        log("   Las cargas del Excel ya incluyen el peso propio (no se suma de nuevo).")
    log(f"   {'Combo':<7}{'Pu[t]':>8}{'Mux[t·m]':>10}{'Muy[t·m]':>10}"
        f"{'qmax[t/m²]':>12}{'Contacto%':>11}  Estado")
    for c, cp in zip(r["combos"], r["campos"]):
        log(f"   {c['nombre']:<7}{c['P']:8.2f}{cp['Mx']:10.3f}{cp['My']:10.3f}"
            f"{cp['q_max']:12.2f}{cp['contacto']:11.1f}  {texto_estado(cp)}")
    log("")
    log(" 2) FLEXIÓN EN LA CARA DE LA COLUMNA (ACI 13.2.7.1)")
    for e, b, Mu, Asr, Asm, bar, pM, et, cmb in (
            ("X", g["By"], r["Mu_x"], r["As_req_x"], r["As_min_x"], r["bar_x"],
             r["phiMn_x"], r["eps_t_x"], r["combo_flex_x"]),
            ("Y", g["Bx"], r["Mu_y"], r["As_req_y"], r["As_min_y"], r["bar_y"],
             r["phiMn_y"], r["eps_t_y"], r["combo_flex_y"])):
        log(f"   Barras paralelas a {e}: Mu = {Mu:.2f} t·m ({cmb}), ancho = {b:.2f} m")
        log(f"     As,req = {Asr:.2f} cm²   As,min = 0.0018·b·{'h' if CRITERIO == 'ACI' else 'd'}"
```

```
            f" = {Asm:.2f} cm²")
        log(f"     Armado: {texto_armado(bar)}  -> As = {bar['As']:.2f} cm²  "
            f"(s,max = {bar['s_max']*100:.0f} cm)")
        log(f"     ϕMn = {pM:.2f} t·m ≥ Mu  ->  D/C = {Mu/pM:.3f}   εt = {et:.4f} "
            f"{'(controlada por tracción)' if et >= 0.005 else ''}")
    log("")
    imprimir_cortantes(r)
    log("")
    log(" 5) VERIFICACIONES COMPLEMENTARIAS")
    a = r["anclaje"]
    log(f"   Anclaje: disponible = {a['disp']*100:.1f} cm, ld = {a['ld']*100:.1f} cm, "
        f"ldh = {a['ldh']*100:.1f} cm -> {a['tipo']}")
    log(f"   Aplastamiento: Pu,max = {r['Pu_max']:.2f} t ≤ ϕBn = {r['phiBn']:.2f} t -> "
        f"{'CUMPLE' if r['Pu_max'] <= r['phiBn'] else 'NO CUMPLE (colocar dowels)'}")
    log(f"   Zapata rectangular: {r['rectangular']}")
    log(f"   Acero superior: {r['acero_superior']}")
    log("")
    nombre, dc, combo = r["gobierna"]
    log(f" Verificación gobernante: {nombre}, D/C = {dc:.3f} ({combo})")
    for a in r["advertencias"]:
        log(f" ADVERTENCIA: {a}")
    log(f" RESULTADO: {r['estado']}" + (f": {r['motivo']}" if r["estado"] != "CUMPLE" else ""))
    log("")


def imprimir_cortantes(r):
    g = r["geo"]
    _, vx_max, _, vy_max = voladizos(g)
    log(" 3) CORTANTE UNIDIRECCIONAL a d de la cara (ACI 8.4.3.2, 22.5.5.1c)")
    for e, vol, d, masa, rho, vc, phiVc, dc, cmb, Vu in (
            ("X", vx_max, r["dx"], r["masa_x"], r["rho_w_x"], r["vc_x"],
             r["phiVc_x"], r["DC_v1_x"], r["DC_v1_x_combo"], r["Vu1_x"]),
            ("Y", vy_max, r["dy"], r["masa_y"], r["rho_w_y"], r["vc_y"],
             r["phiVc_y"], r["DC_v1_y"], r["DC_v1_y_combo"], r["Vu1_y"])):
        log(f"   Dir. {e}: voladizo mayor = {vol*100:.1f} cm,  d = {d*100:.1f} cm")
        if masa:
            log(f"           d ≥ voladizo: la sección a d de la cara queda fuera de la zapata.")
            log(f"           No hay cortante en una dirección -> hormigón en masa.")
        else:
            log(f"           La sección a d queda {(vol - d)*100:.1f} cm dentro del voladizo.")
            log(f"           ρw = {rho:.5f}, vc = 2.12·λs·ρw^(1/3)·√f'c = {vc:.2f} kg/cm², "
                f"ϕVc = {phiVc:.2f} t)")
            log(f"           Vu = {Vu:.2f} t ({cmb})  ->  D/C máx = {dc:.3f}")
    log("")
    log(" 4) CORTANTE BIDIRECCIONAL - PUNZONAMIENTO a d/2 (ACI 22.6)")
    if r["bloque"]:
        log("   No aplica: d ≥ voladizo en ambas direcciones; la superficie de falla a 45°")
        log("   desde la cara de la columna sale por los bordes (hormigón en masa).")
        return
    gob = max(r["detalle"], key=lambda e: e["DC_punz"])
    log(f"   bo = {gob['bo']:.3f} m,  vc = {r['vc_p']:.2f} kg/cm²,  ϕvc = {r['phi_vc_p']:.1f} t/m²")
    if CRITERIO == "CLASE":
        log("   Criterio CLASE: vu = Vu/(bo·d), sin transferencia de momento (Msc solo informativo)")
    log(f"   {'Combo':<7}{'Vu[t]':>8}{'Msc,x':>8}{'Msc,y':>8}{'vu[t/m²]':>10}{'D/C':>7}")
    for e in r["detalle"]:
        log(f"   {e['Combo']:<7}{e['Vu_p']:8.2f}{e['Msc_x']:8.3f}{e['Msc_y']:8.3f}"
            f"{e['vu_p']:10.2f}{e['DC_punz']:7.3f}")


# ============================================================================
# 7. GRÁFICOS
# ============================================================================
def campo_nombre(r):
    return r["combos"][r["idx_punz"]]["nombre"]


def figura_zapata(r):
    """Planta con presiones y secciones críticas, y corte con el armado.
    Usa Figure (sin pyplot) para poder dibujarse también dentro de la interfaz."""
    g = r["geo"]
    campo = r["campos"][r["idx_punz"]]
    Bx, By, cx, cy, ex, ey = g["Bx"], g["By"], g["cx"], g["cy"], g["ex"], g["ey"]
    fig = Figure(figsize=(13, 6))
    ax1, ax2 = fig.subplots(1, 2, gridspec_kw={"width_ratios": [1.15, 1]})
    # --- Planta ------------------------------------------------------------
    im = ax1.pcolormesh(campo["X"], campo["Y"], campo["q"], shading="auto", cmap="YlOrRd")
    fig.colorbar(im, ax=ax1, label="qu [tonf/m²]")
    ax1.add_patch(Rectangle((-Bx / 2, -By / 2), Bx, By, fill=False, lw=2, ec="k"))
    ax1.add_patch(Rectangle((ex - cx / 2, ey - cy / 2), cx, cy, fc="0.55", ec="k", lw=1.5))
    ax1.plot([ex + cx / 2] * 2, [-By / 2, By / 2], "k-", lw=1.2, label="Flexión (cara)")
    ax1.plot([ex - cx / 2] * 2, [-By / 2, By / 2], "k-", lw=1.2)
    ax1.plot([-Bx / 2, Bx / 2], [ey + cy / 2] * 2, "k-", lw=1.2)
    ax1.plot([-Bx / 2, Bx / 2], [ey - cy / 2] * 2, "k-", lw=1.2)
    if r.get("bloque"):
        titulo = f"Zapata P{r['punto']} - Hormigón en masa, qu ({campo_nombre(r)})"
    else:
        for s in (+1, -1):
            xs, ys = ex + s * (cx / 2 + r["dx"]), ey + s * (cy / 2 + r["dy"])
            if abs(xs) < Bx / 2:
                ax1.plot([xs] * 2, [-By / 2, By / 2], "b--", lw=1.2,
                         label="Cortante 1D (d)" if s == 1 else None)
            if abs(ys) < By / 2:
                ax1.plot([-Bx / 2, Bx / 2], [ys] * 2, "b--", lw=1.2)
        b1, b2 = cx + r["d"], cy + r["d"]
        ax1.add_patch(Rectangle((ex - b1 / 2, ey - b2 / 2), b1, b2, fill=False, ec="g", lw=2,
                                ls="-.", label="Punzonamiento (d/2)"))
        titulo = f"Zapata P{r['punto']} - Planta y presión qu ({r['DC_punz_combo']})"
    if r.get("estado") == "REVISAR":
        titulo += "\nREVISAR: pérdida de contacto parcial"
    ax1.set_aspect("equal")
    ax1.set_xlim(-Bx / 2 * 1.05, Bx / 2 * 1.05)
```

```
    ax1.set_ylim(-By / 2 * 1.05, By / 2 * 1.05)
    ax1.set_xlabel("X [m]")
    ax1.set_ylabel("Y [m]")
    ax1.set_title(titulo)
    ax1.legend(loc="upper right", fontsize=8)

    # --- Corte A-A (paralelo a X) ------------------------------------------
    h = r["h"]
    ax2.add_patch(Rectangle((-Bx / 2, 0), Bx, h, fc="0.88", ec="k", lw=1.5))
    ax2.add_patch(Rectangle((ex - cx / 2, h), cx, 0.5, fc="0.6", ec="k", lw=1.5))
    if r.get("bloque"):
        for s in (+1, -1):
            ax2.plot([ex + s * cx / 2, ex + s * (cx / 2 + h)], [h, 0], "g--", lw=1.3,
                     label="Falla a 45° (sale de la zapata)" if s == 1 else None)
    db = r["db"] / 1000.0
    z_inf_x = REC + db / 2 if r["dx"] >= r["dy"] else REC + 1.5 * db
    z_inf_y = REC + 1.5 * db if r["dx"] >= r["dy"] else REC + db / 2
    ax2.plot([-Bx / 2 + REC, Bx / 2 - REC], [z_inf_x] * 2, "r-", lw=2.5,
             label=f"As paralelo a X: {texto_armado(r['bar_x'])}")
    n_y = r["bar_y"]["n"]
    xs = np.linspace(-Bx / 2 + REC + db / 2, Bx / 2 - REC - db / 2, n_y)
    ax2.plot(xs, [z_inf_y] * n_y, "o", color="navy", ms=5,
             label=f"As paralelo a Y: {texto_armado(r['bar_y'])}")
    if r["levantamiento"]:
        ax2.plot([-Bx / 2 + REC, Bx / 2 - REC], [h - REC] * 2, "m-", lw=2,
                 label="Acero superior (levantamiento)")
    ax2.annotate("", xy=(Bx / 2 + 0.08, 0), xytext=(Bx / 2 + 0.08, h),
                 arrowprops=dict(arrowstyle="<->"))
    ax2.text(Bx / 2 + 0.11, h / 2, f"h = {h*100:.0f} cm", va="center")
    ax2.annotate("", xy=(-Bx / 2, -0.08), xytext=(Bx / 2, -0.08),
                 arrowprops=dict(arrowstyle="<->"))
    ax2.text(0, -0.15, f"Bx = {Bx:.2f} m", ha="center")
    ax2.set_aspect("equal")
    ax2.set_xlim(-Bx / 2 - 0.2, Bx / 2 + 0.55)
    ax2.set_ylim(-0.3, h + 0.8)
    ax2.axis("off")
    ax2.set_title("Corte A-A (paralelo a X)")
    ax2.legend(loc="upper center", bbox_to_anchor=(0.5, -0.02), fontsize=8, ncol=1)
    fig.tight_layout()
    return fig


def graficar_zapata(r, ruta):
    figura_zapata(r).savefig(ruta, dpi=150)


def figura_resumen(resultados):
    """Gráfico de barras de la relación demanda/capacidad de todas las zapatas."""
    puntos = [f"P{r['punto']}" for r in resultados]
    series = [("DC_flex_x", "Flexión X"), ("DC_flex_y", "Flexión Y"),
              ("DC_v1_x", "Cortante 1D X"), ("DC_v1_y", "Cortante 1D Y"),
              ("DC_punz", "Punzonamiento")]
    x = np.arange(len(puntos))
    ancho = 0.16
    fig = Figure(figsize=(13, 5))
    ax = fig.subplots()
    for k, (clave, nombre) in enumerate(series):
        ax.bar(x + (k - 2) * ancho, [r.get(clave, np.nan) for r in resultados],
               ancho, label=nombre)
    ax.axhline(1.0, color="r", ls="--", lw=1.5, label="Límite D/C = 1.0")
    ax.set_xticks(x)
    ax.set_xticklabels(puntos)
    ax.set_ylabel("Demanda / Capacidad")
    ax.set_title("Relación demanda/capacidad de las zapatas diseñadas (ACI 318-19)")
    ax.legend(fontsize=8, loc="upper left", bbox_to_anchor=(1.01, 1.0))
    ax.grid(axis="y", alpha=0.3)
    fig.tight_layout()
    return fig


def graficar_resumen(resultados, ruta):
    figura_resumen(resultados).savefig(ruta, dpi=150)


# ============================================================================
# 8. EXPORTACIÓN A EXCEL
# ============================================================================
def dc_texto(valor):
    return "No aplica" if math.isnan(valor) else round(valor, 3)


def exportar_excel(resultados, ruta, datos=None):
    """Excel de resultados: Resumen (tabla de entrega), Detalle_Combos (todas las
    combinaciones), Datos_Entrada (tabla validada) y Parametros."""
    resumen, detalle = [], []
    for r in resultados:
        g = r["geo"]
        fila = {"Punto": r["punto"], "Bx [m]": g["Bx"], "By [m]": g["By"],
                "Columna [cm]": f"{g['cx']*100:.0f}x{g['cy']*100:.0f}",
                "ecx, ecy [cm]": f"{g['ex']*100:g}, {g['ey']*100:g}",
                "f'c [kg/cm²]": FC, "fy [kg/cm²]": FY, "h [cm]": round(r["h"] * 100, 1)}
        if "bar_x" in r:
            ok = lambda c: c["estado"] in ESTADOS_RESUELTOS      # noqa: E731
            validos = [c for c in r["campos"] if ok(c)]
            pz = "No aplica" if r["bloque"] else None
            fila.update({
                "dx [cm]": round(r["dx"] * 100, 1), "dy [cm]": round(r["dy"] * 100, 1),
                "d prom [cm]": round(r["d"] * 100, 1), "Ø [mm]": r["db"],
                "Pu max [t]": r["Pu_max"], "W zapata+relleno [t]": round(r["campos"][0]["W"], 2),
                "qu max [t/m²]": round(max(c["q_max"] for c in validos), 2),
                "Contacto mín [%]": round(min(c["contacto"] for c in validos), 1),
```

```
                # Flexión
                "Mu x [t·m]": round(r["Mu_x"], 2), "Combo flexión X": r["combo_flex_x"],
                "As req x [cm²]": round(r["As_req_x"], 2), "As min x [cm²]": round(r["As_min_x"], 2),
                "As prov x [cm²]": round(r["bar_x"]["As"], 2), "Armado X": texto_armado(r["bar_x"]),
                "φMn x [t·m]": round(r["phiMn_x"], 2), "D/C flexión X": round(r["DC_flex_x"], 3),
                "Mu y [t·m]": round(r["Mu_y"], 2), "Combo flexión Y": r["combo_flex_y"],
                "As req y [cm²]": round(r["As_req_y"], 2), "As min y [cm²]": round(r["As_min_y"], 2),
                "As prov y [cm²]": round(r["bar_y"]["As"], 2), "Armado Y": texto_armado(r["bar_y"]),
                "φMn y [t·m]": round(r["phiMn_y"], 2), "D/C flexión Y": round(r["DC_flex_y"], 3),
                # Cortante unidireccional
                "Vu 1D x [t]": "No aplica" if r["masa_x"] else round(r["Vu1_x"], 2),
                "φVc x [t]": round(r["phiVc_x"], 2), "D/C cortante 1D X": dc_texto(r["DC_v1_x"]),
                "Combo cortante 1D X": r["DC_v1_x_combo"],
                "Vu 1D y [t]": "No aplica" if r["masa_y"] else round(r["Vu1_y"], 2),
                "φVc y [t]": round(r["phiVc_y"], 2), "D/C cortante 1D Y": dc_texto(r["DC_v1_y"]),
                "Combo cortante 1D Y": r["DC_v1_y_combo"],
                # Punzonamiento
                "Vu punz [t]": pz or round(r["Vu_p"], 2),
                "vu punz [t/m²]": pz or round(r["vu_p"], 2),
                "φvc punz [t/m²]": round(r["phi_vc_p"], 2),
                "φVc punz [t]": round(r["phi_vc_p"] * 2 * (g["cx"] + g["cy"] + 2 * r["d"]) * r["d"], 2),
                "D/C punzonamiento": dc_texto(r["DC_punz"]),
                "Combo punzonamiento": r["DC_punz_combo"],
                "Cortante 1D": ("No aplica en X ni Y (hormigón en masa)" if r["bloque"] else
                                "No aplica en X (hormigón en masa)" if r["masa_x"] else
                                "No aplica en Y (hormigón en masa)" if r["masa_y"] else
                                "Se verifica en X e Y"),
                "Verificación gobernante": f"{r['gobierna'][0]} (D/C = {r['gobierna'][1]:.3f})",
                "Combinación controlante": r["gobierna"][2],
                "Controla h": r["control_h"],
                "Anclaje": r["anclaje"]["tipo"],
                "Aplastamiento": "CUMPLE" if r["Pu_max"] <= r["phiBn"] else "NO CUMPLE",
                "Acero superior": r["acero_superior"]})
            for e in r["detalle"]:
                d = {"Punto": r["punto"],
                     "Estado presión": texto_estado(r["campos"][e["idx"]])}
                d.update({k: (round(v, 4) if isinstance(v, float) else v)
                          for k, v in e.items() if k not in ("idx",)})
                if r.get("bloque"):
                    for k in ("Vu1_x", "Vu1_y", "Vu_p", "Msc_x", "Msc_y", "vu_p",
                              "DC_v1_x", "DC_v1_y", "DC_punz", "bo"):
                        d[k] = "No aplica"
                detalle.append(d)
        for c, cp in zip(r["combos"], r["campos"]):
            if cp["estado"] not in ESTADOS_RESUELTOS or "bar_x" not in r:
                detalle.append({"Punto": r["punto"], "Estado presión": texto_estado(cp),
                                "Combo": c["nombre"], "Pu": c["P"], "Mux_base": cp["Mx"],
                                "Muy_base": cp["My"]})
        fila["Advertencias"] = " | ".join(r.get("advertencias", [])) or "—"
        fila["Estado"] = r.get("estado", "NO CUMPLE") + (
            "" if r.get("estado") == "CUMPLE" else f": {r['motivo']}")
        resumen.append(fila)
    parametros = pd.DataFrame({
        "Parámetro": ["f'c [kg/cm²]", "fy [kg/cm²]", "Recubrimiento [m]", "φ flexión",
                      "φ cortante", "λ", "λs (ACI 13.2.6.2)", "αs", "ρ mínima",
                      "h inicial [m]", "h máximo [m]", "Δh [m]", "h inicial por zapata [m]",
                      "Diámetros [mm]", "Separación mínima [m]",
                      "Incluir peso propio", "Factor peso propio", "γc [t/m³]", "γs [t/m³]",
                      "Df [m]", "Traslado de cortante V·h", "Malla", "Criterio",
                      "Diámetro por zapata [mm]"],
        "Valor": [FC, FY, REC, PHI_FLEXION, PHI_CORTE, LAMBDA, LAMBDA_S, ALPHA_S,
                  RHO_MIN, H_MIN, H_MAX, DH, str(H_MIN_PUNTO) if H_MIN_PUNTO else "—",
                  ", ".join(map(str, DIAMETROS_CLASE if CRITERIO == "CLASE" else DIAMETROS)),
                  S_CLASE if CRITERIO == "CLASE" else S_MIN,
                  INCLUIR_PESO_PROPIO, FACTOR_PESO, GAMMA_C, GAMMA_S, DF,
                  TRASLADAR_CORTANTE, f"{N_MALLA}x{N_MALLA}", texto_criterio(),
                  str(DIAMETRO_PUNTO) if DIAMETRO_PUNTO else "—"]})
    with pd.ExcelWriter(ruta, engine="openpyxl") as wr:
        pd.DataFrame(resumen).to_excel(wr, sheet_name="Resumen", index=False)
        pd.DataFrame(detalle).to_excel(wr, sheet_name="Detalle_Combos", index=False)
        if datos is not None:
            datos.to_excel(wr, sheet_name="Datos_Entrada", index=False)
        parametros.to_excel(wr, sheet_name="Parametros", index=False)
        for hoja in wr.sheets.values():
            for col in hoja.columns:
                largo = max(len(str(c.value)) if c.value is not None else 0 for c in col)
                hoja.column_dimensions[col[0].column_letter].width = min(largo + 2, 60)


# ============================================================================
# 9. PROGRAMA PRINCIPAL
# ============================================================================
def main(progreso=None):
    """Diseña todas las zapatas del Excel y guarda los resultados.
    progreso(k, n, punto): función opcional que se llama al terminar cada zapata
    (la usa la interfaz gráfica)."""
    MEMORIA.clear()
    validar_parametros()
    df = leer_datos(ARCHIVO_EXCEL, HOJA_EXCEL)
    os.makedirs(CARPETA_SALIDA, exist_ok=True)
    log("DISEÑO DE ZAPATAS AISLADAS - ACI 318-19")
    log(f"Archivo: {os.path.basename(ARCHIVO_EXCEL)}  |  {len(df)} filas, "
        f"{df['Point'].nunique()} zapatas, combinaciones: {', '.join(df['Load'].unique())}")
    log(f"f'c = {FC:.0f} kg/cm²   fy = {FY:.0f} kg/cm²   rec = {REC*100:.1f} cm   "
        f"h = {H_MIN*100:.0f} a {H_MAX*100:.0f} cm cada {DH*100:.0f} cm   "
        f"traslado V·h = {TRASLADAR_CORTANTE}")
    log(f"Peso propio de zapata y relleno: "
        f"{f'incluido (factor {FACTOR_PESO:g}, Df = {DF:g} m)' if INCLUIR_PESO_PROPIO else 'ya incluido en las cargas'}")
    log(f"Criterio: {texto_criterio()}")
    if DIAMETRO_PUNTO:
        log(f"Diámetro impuesto por zapata [mm]: {DIAMETRO_PUNTO}")
```

```
    log("")

    resultados = []
    grupos = list(df.groupby("Point", sort=True))
    for k, (punto, datos) in enumerate(grupos, start=1):
        r = disenar_zapata(punto, datos)
        inicio = len(MEMORIA)
        imprimir_zapata(r)
        r["memoria"] = "\n".join(MEMORIA[inicio:])
        if "bar_x" in r:
            graficar_zapata(r, os.path.join(CARPETA_SALIDA, f"Zapata_P{punto}.png"))
        resultados.append(r)
        if progreso:
            progreso(k, len(grupos), punto)

    log("=" * 79)
    log(" RESUMEN GENERAL")
    log("=" * 79)
    log(f" {'Pto':<4}{'BxBy[m]':>10}{'h[cm]':>7}  {'Armado X':<22}{'Armado Y':<22}"
        f"{'D/C máx':>8}  Estado")
    for r in resultados:
        if "bar_x" not in r:
            log(f" {r['punto']:<4} NO DISEÑADA: {r['motivo']}")
            continue
        dc = np.nanmax([r["DC_flex_x"], r["DC_flex_y"], r["DC_v1_x"], r["DC_v1_y"], r["DC_punz"]])
        log(f" {r['punto']:<4}{r['geo']['Bx']:>5.2f}x{r['geo']['By']:<4.2f}{r['h']*100:>7.0f}  "
            f"{texto_armado(r['bar_x']):<22}{texto_armado(r['bar_y']):<22}{dc:8.3f}  "
            f"{r['estado']}"
            f"{'  (hormigón en masa)' if r['bloque'] else ''}")
    for r in resultados:
        for a in r.get("advertencias", []):
            log(f" P{r['punto']}: {a}")

    graficar_resumen([r for r in resultados if "bar_x" in r],
                     os.path.join(CARPETA_SALIDA, "Resumen_Demanda_Capacidad.png"))
    exportar_excel(resultados, os.path.join(CARPETA_SALIDA, "Resultados_Zapatas.xlsx"), df)
    with open(os.path.join(CARPETA_SALIDA, "Memoria_Calculo.txt"), "w", encoding="utf-8") as f:
        f.write("\n".join(MEMORIA))
    print(f"\nResultados guardados en: {CARPETA_SALIDA}")
    return resultados


if __name__ == "__main__":
    try:
        resultados = main()
    except ErrorDatos as error:
        print(f"\n*** {error}")
        raise SystemExit(1)
```